



**EASTERN SAMAR
STATE UNIVERSITY**
Excellence • Accountability • Service



IOT-BASED SMART LABORATORY MANAGEMENT SYSTEM FOR ESSU ARTECHE CAMPUS

An Undergraduate Capstone Project

Presented to the Faculty of the
DEPARTMENT OF INFORMATION TECHNOLOGY
College of Computer Studies
Eastern Samar State University
Arteche, Eastern Samar State University
Arteche, Eastern Samar

In Partial Fulfillment of the Course Requirement for the Degree
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

**ACASO, CHARLIE
BERTOS, MATILDE
MAISO, RUEL
POMAREJOS, STEPHANIE
TOMBADO, BERLI**

APRIL 2026



Chapter I

INTRODUCTION

Background and Rationale

In recent years, the continuous growth of information technology has significantly influenced higher education, leading to more connected and technology-supported learning environments worldwide. One of the most notable developments is the use of the Internet of Things (IoT) in academic settings, where devices and systems are interconnected to allow real-time data collection, monitoring, and automation. This concept is commonly seen in “smart campuses,” where technology is used to improve both teaching and administrative processes (Mohanty et al., 2024). As a result, computer laboratories and classrooms have become more dependent on digital tools, especially in programs that require hands-on computer use such as Information Technology.

Several studies have explored how these technologies can improve classroom and laboratory management. According to Mohanty et al. (2024), IoT-enabled environments allow instructors to monitor student activities in real time, making it easier to provide immediate support and improve student engagement. Similarly, Angamuthu et al. (2025) presented a LAN-based monitoring system that enables centralized tracking of workstation activities, showing how such systems can simplify supervision in computer laboratories. These studies suggest that integrating monitoring and control systems can help address common challenges in managing computer-based learning environments.

Despite these developments, existing laboratory management systems still show several limitations. Many available solutions focus only on specific functions, such as screen monitoring, access restriction, or scheduling, but fail to combine these features into a single system. Some systems provide monitoring without control, while others offer scheduling without real-time hardware status updates. To the best of the researchers’ knowledge, there is still no unified system that integrates multi-computer screen monitoring, centralized command-based control, session-based time management, and hardware monitoring within a single LAN-based platform. This lack of integration creates a gap where current systems remain fragmented and unable to fully support an efficient and organized smart laboratory environment. In the Philippines, higher education



institutions have started adopting digital technologies to improve learning and operations. However, many campuses still face challenges such as limited resources, insufficient technical support, and inconsistent network infrastructure. These limitations affect the implementation of more advanced laboratory systems. As noted by Bolasco et al. (2025), some institutions continue to rely on manual supervision and separate tools, which often lead to delays, inefficiencies, and a higher risk of human error. This situation is also evident at Eastern Samar State University (ESSU) Arteche Campus, where laboratory management is still largely done manually. Instructors supervise students directly, check workstation conditions individually, and record usage through logbooks. These practices are time-consuming and may lead to missed issues or inaccurate records, especially during busy laboratory sessions.

In response to these challenges, this study aims to design, implement, and develop an IoT-based Smart Laboratory Management System tailored for ESSU Arteche Campus. The proposed system will operate within the campus local area network and provide a centralized interface where instructors can monitor student workstation screens in real time, send control commands, manage session time, and view basic hardware status through connected sensors. The system is designed to match the actual needs and limitations of the campus while improving current laboratory practices.

The project is expected to provide value by making laboratory management more organized, efficient, and easier to supervise. It can help instructors manage multiple workstations more effectively while reducing reliance on manual processes. In addition, the system may serve as a practical reference for other institutions with similar conditions that are looking for a simple and cost-effective way to implement IoT-based laboratory management solutions.

OBJECTIVES OF THE STUDY

The general objective of this study is to develop and evaluate an IoT-based smart laboratory management system for ESSU Arteche Campus that enables real-time monitoring, control, and supervised usage of computer laboratory workstations within a local area network.

1. The specific objectives aim to:
 - 1.1. Develop a LAN-based real-time screen monitoring system that allows instructors to view and supervise multiple student workstations through a centralized dashboard.
 - 1.2. Implement workstation access control mechanisms, including administrative control and session-based timers, to regulate workstation usage during laboratory sessions.
 - 1.3. Incorporate IoT-based workstation health monitoring to collect and display temperature and fan status data for each workstation in support of equipment safety awareness.
 - 1.4. Establish a centralized laboratory management interface that provides a consolidated real-time view of workstation activity and hardware status.
2. Evaluate the developed system using ISO/IEC 25010 software quality standards, assessing characteristics such as:
 - 2.1. Functional suitability
 - 2.2. Performance efficiency
 - 2.3. Compatibility
 - 2.4. Interaction capability
 - 2.5. Reliability
 - 2.6. Security
 - 2.7. Maintainability
 - 2.8. Flexibility
 - 2.9. Safety

SCOPE AND LIMITATIONS

This study focuses on the design, development, and local deployment of an IoT-based smart laboratory management system for the Bachelor of Science in Information Technology program at Eastern Samar State University (ESSU) Arteche Campus, particularly within the College of Computer Studies. The system is intended to be used in a controlled computer laboratory environment and is designed to support regular instructional activities during laboratory sessions. It includes the development of both instructor and student applications. On the student side, each workstation is equipped with client software that enables real-time screen monitoring, receives command-based control instructions from the instructor, participates in session-based time management, and transmits basic hardware status data through integrated IoT sensors. On the instructor side, the system provides a centralized dashboard that allows real-time monitoring of student activities, command execution, session management, and visualization of workstation conditions. The system operates exclusively within the campus local area network (LAN) and is strictly intended for on-premises use without reliance on external cloud services. Additionally, the study covers the development of user-friendly graphical interfaces and the integration of IoT-based sensor modules to enhance real-time awareness of workstation conditions. All system features were reviewed and refined based on recommendations from academic panel members to ensure alignment with the actual needs and technical constraints of ESSU Arteche Campus.

However, the study has several limitations in terms of scope, technical capabilities, and implementation. The proposed system is limited to manually configured computers within a single laboratory and does not support automatic device detection, dynamic enrollment, or multi-laboratory deployment. It operates strictly within a local area network (LAN) and is intended for on-premises use only, without cloud integration or remote access functionality. In terms of security, the system implements basic user authentication and session control suitable for a controlled academic environment. Advanced security features such as role-based access control, comprehensive data encryption, and network intrusion protection are not included, and security evaluation is limited to system integrity and controlled access based on ISO/IEC 25010 standards. Functionally, the system focuses on real-time monitoring and control during laboratory sessions. While it provides session logging and basic usage tracking, it does not include advanced analytics, automated reporting, or predictive maintenance. Workstation monitoring is limited to real-



time sensor data visualization without automated diagnostics or predictive analysis. Overall, the system is developed as an academic prototype for controlled laboratory use, and further enhancements are needed to support large-scale deployment, advanced security, and full commercial implementation.

SIGNIFICANCE OF THE STUDY

This study is significant as it introduces an IoT-based smart laboratory management system that improves efficiency, supervision, and awareness in computer laboratory activities. By integrating real-time monitoring and basic workstation status tracking, it supports better instructional control and decision-making. The significance of the study extends to the following stakeholders:

Eastern Samar State University. The university benefits by enhancing its laboratory facilities with a modern system that improves management and monitoring of computer labs, aligning with its educational mission.

College of Computer Studies. The college gains a more efficient platform for teaching and supervising practical IT courses, helping maintain orderly lab sessions and demonstrating the university's commitment to technology-enhanced learning.

Instructors and Laboratory Administrators. Faculty members and lab staff can more effectively supervise student activity in real time, quickly address technical issues (e.g., high temperatures or unauthorized activity), and manage lab sessions (such as locking unused computers), thereby improving instructional efficiency and student safety.

Students. A well-monitored laboratory environment promotes focused, safe, and reliable learning experiences, minimizing disruptions caused by unmanaged workstation use or undetected hardware conditions.

Project Researchers. The student researchers who developed this system gain practical experience in software and IoT system development, reinforcing their academic learning and enhancing their technical skills; the project also fulfills their graduation requirements.

Future Researchers and Developers. This study provides a reference framework for future IoT-enabled lab management solutions. Researchers and developers can build upon the design and implementation approaches presented here to develop improved or expanded systems for similar educational settings.



Definition of Terms

Access Control. Access control is the process of determining whether a subject (e.g., a user or device) should be granted or denied access to a resource or object.

Client-Server Architecture. A system design in which multiple clients request services or resources from a central server; the server handles processing, data storage, and resource management, while clients handle user interaction.

Dashboard. A type of graphical user interface that provides at-a-glance views of data relevant to a particular objective or process.

Graphical User Interface (GUI). A form of user interface that allows users to interact with electronic devices through graphical icons and visual indicators.

Internet of Things (IoT). A network of physical objects embedded with sensors, software, and other technologies that connect and exchange data over the Internet or other communication networks.

JavaScript Object Notation (JSON). An open standard file format and data interchange format using human-readable text to store and transmit structured data objects consisting of attribute-value pairs and arrays.

Local Area Network (LAN). A computer network that interconnects devices and computers within a limited area (such as a campus or building), enabling high-speed communication and resource sharing.

On-Premises System. A computing system installed and operated within the physical premises of an organization, rather than relying on remote cloud infrastructure.

Sensor. A device or subsystem that detects events or changes in its environment and sends the resulting data to other electronics or systems for processing.

Session-Based Timer. A feature that enforces predetermined time limits on user sessions, automatically terminating or alerting when the allocated session time has elapsed.

Smart Laboratory Management System. An integrated application that uses networked technologies (such as IoT and real-time monitoring) to support supervision, control, and management of laboratory workstations and activities.

Telemetry. The automated process of collecting and transmitting measurements or data from remote sources to a central system for monitoring, analysis, or recording.

Workstation. A high-performance computer designed for technical or scientific applications, intended for use by a single user.



Chapter II

RELATED STUDIES AND RELATED SYSTEMS

The integration of Internet of Things (IoT) technologies into educational environments has been explored to enhance monitoring and control of classroom activities. For example, Mohanty et al. (2024) note that a high-speed network and a secure Local Area Network (LAN) are essential to support the real-time data transmission required for monitoring students and devices in a smart classroom. In the current practice, many educational monitoring systems indeed rely on local network connectivity to provide immediate oversight, as opposed to cloud-dependent solutions. Despite the promise of IoT for education, existing solutions tend to be specialized and scattered: some focus on attendance or scheduling, others on hardware statistics or screen viewing, but none unify all functions in a single LAN-based platform. This chapter reviews relevant research and systems, focusing on those that align with our LAN-based smart lab features: multi-screen monitoring, centralized control, session timing, and per-workstation hardware monitoring.

Related Studies

In global research, IoT and networked systems have been applied to laboratories and classrooms, though often with different emphases. Czekalski et al. (2025) describe a next-generation remote IoT laboratory for Industry 4.0 education. Their system uses open-source software and cloud services to connect students to physical lab equipment from anywhere. It emphasizes connectivity, security, and user management but relies on Internet/cloud infrastructure. In contrast, Alagasan et al. (2021) provide a survey of advanced attendance and monitoring systems. This review highlights technologies like fingerprint, iris recognition, and RFID for tracking people. It shows how IoT (e.g. RFID sensors) can automate attendance logging, improving accuracy and security. However, it focuses on identity verification rather than computer lab control; it does not address multi-PC monitoring or teacher commands.

One study that comes closer to our scope is LabTrack (Bolasco et al., 2025). LabTrack implements a computer lab attendance and monitoring system in a university setting. Students authenticate via RFID or login, which automatically records their entry



into a central database. The administrator can then monitor attendance logs and computer usage in real time. This three-tier system boosts attendance accuracy and security by linking each login to a verified student and timestamp. It demonstrates session-based management and centralized data collection. However, LabTrack focuses on logging and does not provide real-time screen viewing or hardware metrics; it mainly ensures the “who-when” of lab usage.

Within the Philippines, published research on comprehensive lab management systems is scarce. Some local projects emphasize environmental and power management in computer labs (for example, IoT systems to control lighting or power save modes) rather than user monitoring and control. The LabTrack study by Bolasco et al. is one of the few local references addressing computer lab management. It shows that Filipino researchers have implemented automated attendance tracking, but like global systems, it is limited to logging user sessions. No national study was found that combines screen monitoring, computer control, timed sessions, and hardware sensing into a single LAN-based platform. This suggests a gap in local research on integrated lab management systems.

Locally, universities have demonstrated interest in smart labs (for example, USTP’s IoT Laboratory for training), but these efforts typically support IoT education or sensor experiments rather than integrated lab supervision. No published or widely reported system in our region fully meets the listed requirements. In practice, instructors might use separate tools (for instance, one tool for locking PCs and another for taking attendance), but no single solution offers all functions on a secure LAN. Thus, both global and national trends indicate a fragmentation of lab management capabilities: attendance tracking systems exist, and classroom monitoring tools exist, but nothing unifies them.

Related Systems

A variety of commercial and open-source classroom management systems provide some of the needed features, especially screen monitoring and control. For instance, LanSchool is a widely used software that gives teachers real-time visibility into student workstations. It displays thumbnails of all student screens, allowing the teacher to view any student’s desktop in detail. Teachers can also blank (lock) a student’s screen or push a website or application to students. LanSchool operates over the LAN, communicating between teacher and student clients. Its strength is real-time multi-screen monitoring and simple control (blanking or launching apps), which directly supports

classroom focus. Its limitation is that it does not monitor hardware metrics or enforce session timing it is designed purely for guidance and discipline.

NetSupport School is another comprehensive classroom tool with similar capabilities. In NetSupport, an instructor can monitor student screens including dual-monitor setups. The software shows which applications and websites each student is using in real time. Teachers can remotely control a student's PC to assist them or to lock it down. NetSupport also provides a "Blank Screen" feature to instantaneously blank or lock all selected students' screens and disable their input devices. A notable feature is that the student toolbar displays session information, including a countdown or "time remaining" for a task. In other words, NetSupport includes a basic session timer visible to students. These capabilities (multi-screen view, locking screens, application restrictions, and session timers) align closely with our system's requirements for monitoring and control. On the downside, like LanSchool it is focused on user activity and does not natively monitor PC hardware health (temperature, fan speed, etc.).

Faronics Insight is a teacher-led lab control software. Its teacher console runs on Windows and connects to student clients over the LAN. Insight allows remote control of individual student computers (the teacher can directly operate a student's PC) and can even disable the student's keyboard and mouse during control. Teachers can broadcast their screen to all students or vice versa, and can lock/blank screens with one click. Insight also features digital tests and polls. These functions demonstrate centralized command and monitoring: an instructor has oversight of all PCs and can intervene instantly. The limitation is that Insight is proprietary and primarily concerned with software control; it does not gather hardware sensor data or operate purely on a private LAN (it may use cloud services).

The open-source tool Veyon (formerly iTalc) provides similar classroom control capabilities without licensing costs. Veyon enables a teacher to see all student screens in an icon view and access any machine with one click. It can take screenshots on demand and lock all student computers (blocking keyboard/mouse) via a single action. As a free tool, it is flexible and cross-platform. However, it shares the same gap as the others: Veyon manages screens and inputs but does not include IoT sensors or timers by default.

Outside of dedicated classroom tools, lab usage tracking software offers a different approach. For example, LabStats provides software that runs an agent/client/student on each PC and collects usage data. According to LabStats documentation, their system "automatically tracks what is happening in a computer lab"



by installing a client on each machine, which then reports hardware and software activity to an administrator. It can generate reports on CPU and application usage over time. This illustrates workstation-level monitoring (similar to our hardware monitoring requirement), but LabStats is used mostly for analytics and planning (passive tracking), not for real-time teacher control. It lacks features like locking screens or countdown timers.

In summary, current systems cover parts of our needs. Classroom management suites (LanSchool, NetSupport, Veyon, Insight) offer robust multi-PC monitoring and control. Attendance or session systems (LabTrack) provide login logging and usage tracking. Hardware monitoring tools (like LabStats or Angamuthu's LAN Monitor) supply real-time system metrics. However, no single solution unifies all four core elements (monitor, control, timer, hardware) into one LAN-exclusive platform.

Synthesis

Across all reviewed systems, some common features and limitations emerge. Most monitoring tools allow teachers to view student screens simultaneously. In parallel, teachers can often blank or lock those screens to refocus attention. Remote control is also common: instructors can take over or lock individual computers. These features support real-time classroom oversight. On the other hand, systems focused on hardware or usage monitoring (such as LabStats or the LAN Monitor tool by Angamuthu) collect CPU/memory/disk data but do not interactively manage PCs. Session timing is rarely integrated; NetSupport's student toolbar shows a countdown, but most tools lack an enforced timer or scheduling mechanism. Importantly, each reviewed system operates on a local network (LAN) but none restrict themselves to only LAN (some require internet for updates or cloud backends) or incorporate IoT sensors for PC internals.

In effect, the systems are fragmented. Some emphasize monitoring screens and applications. Others emphasize centralized control commands (lock/unlock, remote input). A few handle attendance/session loggings. Very few address workstation hardware health; Angamuthu's tool monitors performance metrics, and usage trackers report activity, but none monitor fan speeds or temperatures as envisioned. None of the existing solutions ties these threads together. This comparison clearly shows that current lab management tools are specialized: each covers one or two aspects (monitoring or control, usage or scheduling) but no single platform offers a unified dashboard that includes real-time screen feeds, instructor commands, session timers, and IoT-based hardware monitoring on a closed LAN.



Research Gap

From the above review, a clear gap is identified: no existing system provides a fully integrated LAN-based solution combining all key features. In other words, current literature and products do not present a single platform for an IoT-enabled smart lab that simultaneously delivers real-time multi-PC screen monitoring, centralized lock/unlock and control commands, session-based time management, and workstation-level hardware monitoring without relying on cloud services. The proposed LAN-Based IoT Smart Laboratory Management System will address this gap by unifying these components into one local dashboard, ensuring cohesive management and monitoring of a computer laboratory environment.



CHAPTER III

METHODOLOGY

This chapter presents the methodological framework used to plan and design the proposed IoT-Based Smart Laboratory Management System for ESSU Arteche Campus. It describes the chosen development model, feasibility analyses, process design, interface design, and the development and evaluation environments.

Software Development Model

This study adopts the Prototyping Model as the software development approach for the proposed IoT-Based Smart Laboratory Management System for ESSU Arteche Campus. The model was selected because the system integrates interactive monitoring dashboards, graphical user interfaces, and IoT-based hardware components that require early visualization and iterative refinement throughout the development lifecycle. The Prototyping Model emphasizes the development of preliminary working versions of the system in order to clarify requirements and evaluate system behavior before the final design is completed.

In this approach, early artifacts such as interface wireframes, system diagrams, process models, and conceptual hardware layouts are developed to represent the system's structure and functionality. These preliminary representations allow the researchers to visualize system behavior and verify whether the identified functional and non-functional requirements are properly addressed. Through iterative refinement and evaluation, the prototype gradually evolves until the system design becomes stable and ready for final implementation.

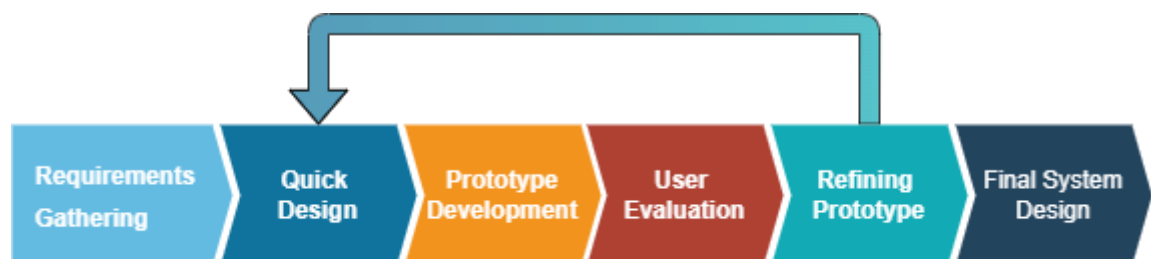


Figure 1. Prototyping Model



Figure 1 presents the Prototyping Model adopted in the development of the proposed system. The development process follows several stages including requirements gathering, quick design, prototype development, user evaluation, prototype refinement, and final system design.

During the requirements gathering phase, the researchers analyzed existing laboratory supervision practices at ESSU Arteche Campus and identified system requirements such as real-time workstation monitoring, session management, and hardware telemetry integration. This phase ensured that both functional and non-functional requirements were clearly defined and aligned with the needs of the laboratory environment.

This was followed by the quick design stage, where preliminary system representations including interface wireframes, system diagrams, and conceptual hardware layouts were created to visualize the structure of the proposed solution. These initial designs served as a guide for system development and helped clarify how different components would interact within the system.

During the prototype development stage, the initial working version of the system was implemented, which included the teacher monitoring dashboard, student application, and ESP32-based telemetry module. This stage focused on developing a functional model that demonstrates the system's core features and operational flow.

The developed prototype was then subjected to user evaluation, where feedback from the project adviser and selected instructors through consultation was gathered to assess usability, functionality, and overall system performance. This evaluation provided insights into system strengths and areas that required improvement. User evaluation during prototyping involved consultations with the project adviser and selected instructors, while formal system evaluation involved BSIT students and IT professors.

Based on the evaluation results, the system underwent prototype refinement, where improvements were implemented to enhance system functionality, interface usability, and overall system stability. Adjustments were made iteratively to address identified issues and optimize system performance.

Finally, the final system design stage consolidated all refinements into the completed system architecture and documentation, preparing the IoT-Based Smart Laboratory Management System for implementation and evaluation. At this stage, the system components and interactions were finalized in preparation for presentation and validation.



The iterative nature of the Prototyping Model supports gradual refinement of both software and hardware components. Given that the proposed system combines real-time screen monitoring, workstation control mechanisms, and ESP32-based telemetry integration within a local area network environment, early validation through prototype development reduces design ambiguity and technical risk. Each refinement cycle strengthens requirement clarity and improves system representation prior to finalization of the complete system design. Furthermore, this approach allows continuous alignment between system development and user expectations, ensuring that the final system remains relevant and functional within the intended laboratory context. The adoption of the Prototyping Model is appropriate for this study as it supports structured documentation while allowing controlled iterative improvement within an academic system development context.

Planning

The planning phase establishes the foundational framework for the proposed system. It involves analyzing the problem domain, laboratory workflows, and existing infrastructure constraints. During planning, the research team identifies the system's functional and non-functional requirements, constraints, and required resources for the IoT solution. This phase ensures that the system's scope and objectives align with the needs of ESSU Arteche Campus and that any potential technical or logistical issues are anticipated.

Requirements Specifications

The requirements specification phase identifies and defines the detailed functional requirements of the system. Examples include real-time workstation monitoring, user authentication, and data logging. Non-functional requirements including performance targets (acceptable latency), security constraints (authentication and transport security), and usability goals are also documented to guide design and testing. These requirements inform the system's design and form the basis for feasibility studies and subsequent design activities.

Operational Feasibility

Operational feasibility determines whether the proposed system can be effectively implemented within the existing laboratory environment and workflow. In the current setup at ESSU Arteche Campus, laboratory supervision relies on manual observation and the use of disconnected tools, which limits real-time monitoring and delays response to

student activities. To analyze these operational challenges, a Fishbone (Ishikawa) diagram was used to identify the root causes of inefficient and fragmented laboratory supervision.

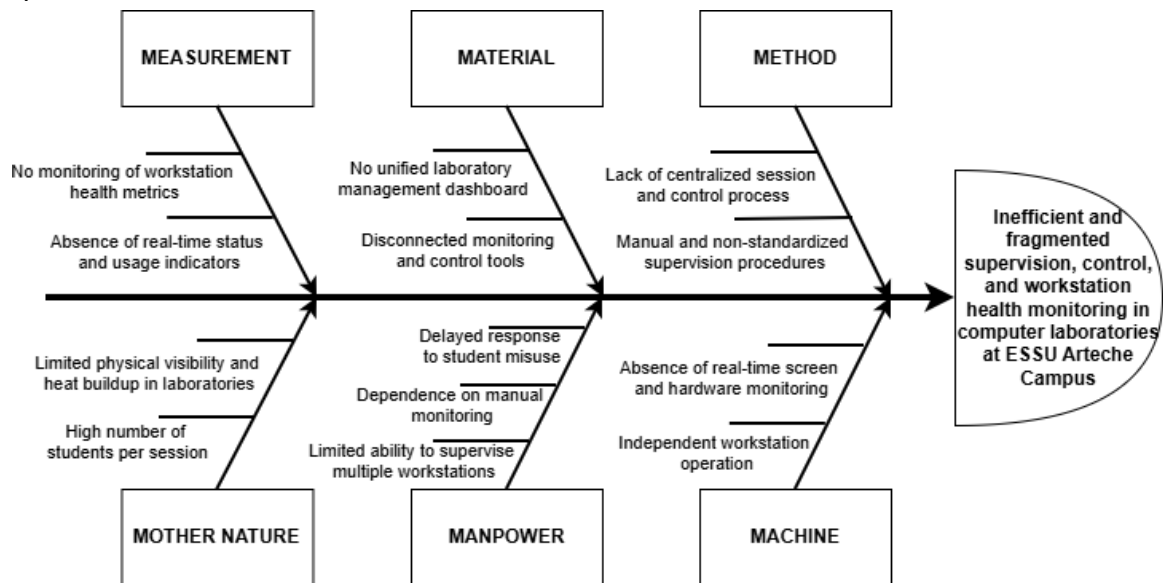


Figure 2. Fishbone (Ishikawa) diagram

The Fishbone (Ishikawa) diagram identifies the root causes of inefficient and fragmented laboratory supervision by grouping them into six key factors. In terms of manpower, supervision relies heavily on instructors, resulting in delayed responses, dependence on manual monitoring, and difficulty managing multiple workstations simultaneously. In terms of method, there is a lack of centralized session management and standardized supervision procedures, leading to inconsistent laboratory operations. From the machine perspective, workstations operate independently without integration, and there is no real-time screen or hardware monitoring system to track activity and status. In measurement, the absence of workstation health metrics and real-time usage indicators makes it difficult to assess performance and monitor student activity. In terms of material, the lack of a unified laboratory management dashboard and the use of disconnected monitoring tools contribute to inefficient system control. Lastly, in terms of environment, physical limitations such as limited visibility, heat buildup, and a high number of students per session further complicate effective supervision.

Based on this analysis, the proposed system is operationally feasible as it introduces centralized monitoring and control within the existing laboratory setup. It supports current instructional practices while improving supervision efficiency without requiring major changes to infrastructure or workflow.



Technical Feasibility

Technical feasibility evaluates whether the available technology and infrastructure can support the proposed system. The study examines the suitability of hardware such as ESP32 microcontrollers and sensors, as well as software platforms including monitoring applications and network protocols. It considers device compatibility, network bandwidth, and data processing capability. The selected technologies, as presented in Tables 1-3, were assessed to ensure they meet system requirements within a local area network environment.

Developmental

Table 1. Technical Requirements (During Development)

Hardware	Software	Database
Desktop/Laptop (e.g. DELL Inspiron 15)	Visual Studio Code	SQLite
ESP32 Microcontroller (WROOM-32)	Arduino IDE	
Laboratory Workstation PCs	Python Interpreter	
	Draw.io, EasyEDA	

Table 1 presents the development tools used in building the system, including both hardware and software components utilized during prototyping. These resources are sufficient to support system development, testing, and initial implementation.

Table 2. Technical Requirements: Hardware (Operational Environment)

Hardware	Minimum Configuration	Recommended Configuration
Teacher/Admin Monitoring Workstation	64-bit PC, Intel Core i3 (or equivalent), 8 GB RAM, 1280×800 display	Intel Core i5 or higher, 8-16 GB RAM, 1920×1080 display
Student Workstation	64-bit PC, Intel Core i3 (or equivalent), 4 GB RAM	Intel Core i3 or higher, 8 GB RAM
ESP32 Microcontroller	ESP32-WROOM-32 development board	ESP32-WROOM-32 (latest revision)



Temperature Sensor	DS18B20 digital temperature sensor	DS18B20 waterproof sensor
Hall Effect Sensor	Digital hall effect sensor module	Hall-effect sensor module with stable output
Prototype Components	Breadboard, jumper wires, 4.7 k Ω pull-up resistor	Soldered or PCB-mounted connections
Power Supply (ESP32)	5V regulated power (USB or adapter)	Dedicated 5V regulated power adapter
Network Infrastructure	Existing LAN router, switch (100 Mbps) and UTP Cables	Gigabit LAN router, switch and Ethernet Cable

Table 2 lists the minimum and recommended hardware specifications required for deploying the system. These include the monitoring workstation, student workstations, ESP32 microcontroller, sensors, and network infrastructure needed to support real-time monitoring within the local area network.

Table 3. Technical Requirements: Software (Operational Environment)

Software	Minimum Requirements	Recommended Requirements
Operating System	Windows 10 (64-bit)	Windows 10/11 (64-bit)
Python Runtime Environment	Python 3.8	Latest Python 3.13 stable
ESP32 Board Support Package	Installed ESP32 board package	Latest stable ESP32 board package

Table 3 presents the software requirements necessary for system operation. It includes the operating systems, runtime environments, and development packages required to run the monitoring system. The specified versions (e.g., Windows 10/11, Python 3.13, and the latest ESP32 board package) ensure system compatibility, stability, and optimal performance.



Schedule Feasibility

Schedule feasibility assesses whether the project can be completed within the available time frame. A Gantt chart is used to present the project activities and their corresponding schedule. The phases of the prototyping process are distributed across a weekly timeline to ensure sufficient time for design, development, and evaluation.

Figure Definition:

Symbols	Description
 Vertical line	Represents the months and their duration
 Horizontal line	Represents the borders of the activities
 Shaded Box	Represents an activity that has already been done
 Unshaded Box	Represents a portion of an activity that has not yet been done



Table 4. Project Gantt Chart

Activities	December 2025				January 2026				February 2026				March 2026				April 2026			
	W 1	W 2	W 3	W 4	W 1	W 2	W 3	W 4	W 1	W 2	W 3	W 4	W 1	W 2	W 3	W 4	W 1	W 2	W 3	W 4
Requirement Identification																				
System Analysis																				
Quick Design																				
Initial Prototype Development																				
User Evaluation																				
Prototype Refinement																				
Final System Design																				
Documentation Finalization																				
Final Review																				

W1, W2, W3, and W4 represent Weeks 1, 2, 3, and 4 of each month

Table 4 shows the project timeline from December 2025 to April 2026, following an iterative prototyping model where the system is gradually developed and refined. The project begins with Requirement Identification (December Weeks 2-4 to January Weeks 1-3), during which laboratory workflows are observed and system requirements are defined. This is followed by System Analysis (January Weeks 2-4), which evaluates existing processes and determines appropriate IoT-based solutions. In February (Weeks 1- 4), the Quick Design phase produces initial system representations such as interface wireframes and conceptual diagrams. Initial Prototype Development then proceeds from February (Weeks 1-4) through early March (Weeks 1-2), during which the core monitoring system components are implemented. Concurrently, User Evaluation sessions are conducted (February Weeks 1 and 3; March Week 2) with the project adviser and selected instructors to assess the system's usability and functionality. Feedback from these evaluations is incorporated during Prototype Refinement (March Week 3). Finally, the project concludes with the Final System Design (March Week 4 to April Week 1), where all components are finalized and prepared for presentation and defense.

DESIGN

This section outlines the design specifications of the proposed system, including user interface considerations, process design, data flow, and system structure.

OUTPUT AND USER INTERFACE/GUI

This subsection presents the proposed graphical user interface design, which provides instructors with a centralized and intuitive environment for monitoring, controlling, and managing laboratory workstations.

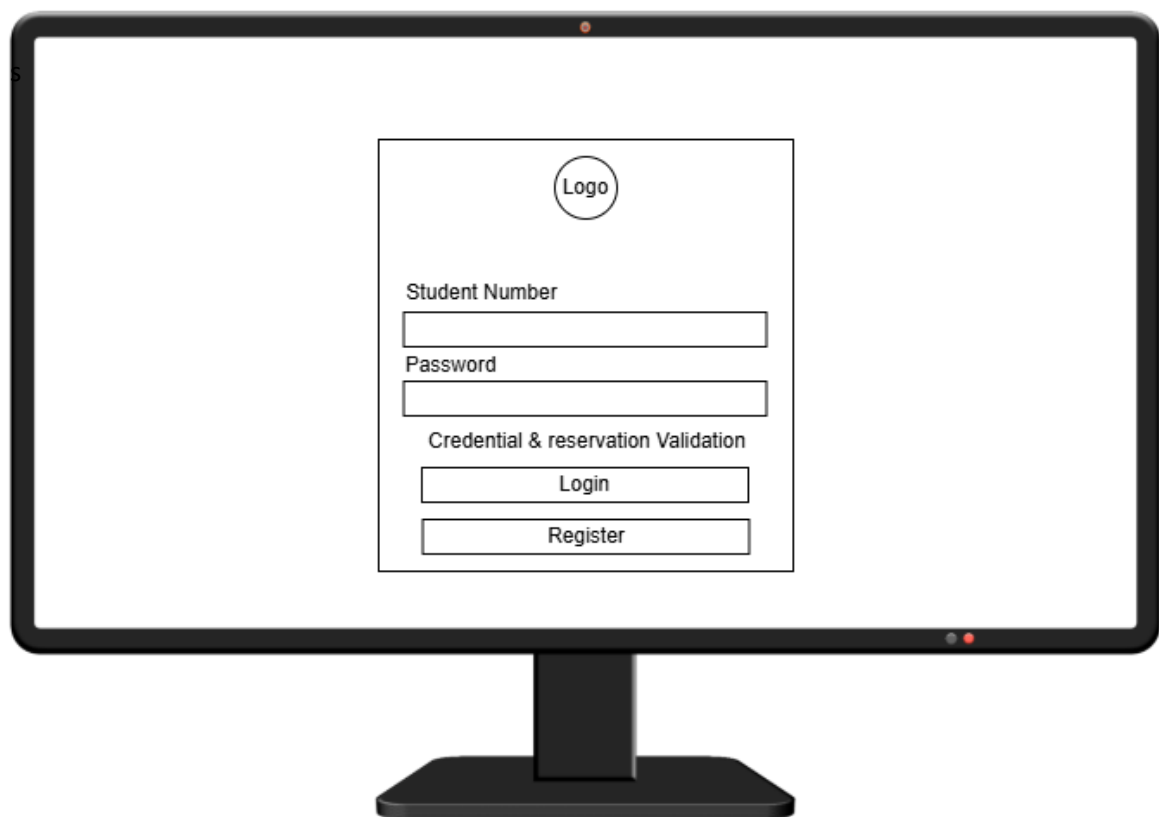


Figure 3. Student Login Interface of the Student PC Application

The figure shows the login interface of the student application, where users enter their student number and password for authentication. It also includes options for login and registration, along with credential and reservation validation to control access to the system.



The image shows a computer monitor displaying a web-based registration interface. The interface is centered on the screen and contains the following elements:

- A circular placeholder labeled "Logo" at the top.
- A text input field for "Full Name".
- A text input field for "Year & Section".
- A text input field for "Student Number".
- A text input field for "Password".
- A label "Registration response" below the password field.
- A button labeled "Register".
- A button labeled "Back To Login".

Figure 4. Student Registration Interface of the Student PC Application

The figure shows the registration interface where students input personal and account details such as full name, year and section, student number, and password. It includes options to submit registration and return to the login screen.



Figure 5. Lock Screen and Chat Interface of the Student PC

The figure shows the lock screen interface displayed during restricted sessions, along with a chat feature that allows communication between the students and the admin operator. It also includes a session timer indicator for monitoring session duration.

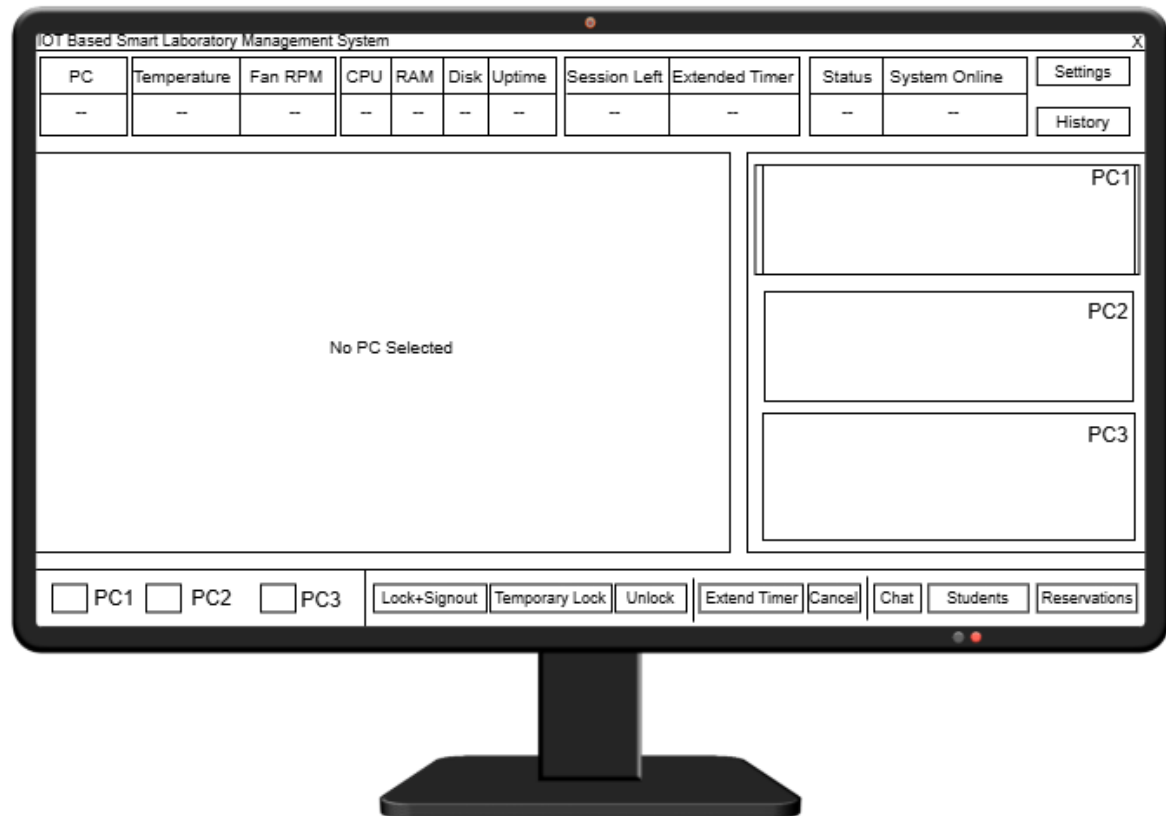


Figure 6. Admin Monitoring Dashboard of the IoT-Based Smart Laboratory Management System

The figure presents the admin monitoring dashboard, which provides a centralized interface for supervising multiple workstations in real time. The top panel displays system and hardware status indicators. The main area shows the 3 selected workstation's live monitoring view, while the side panel displays multiple workstation previews. The bottom section contains control functions including lock, unlock, session management, timer extension, chat, and access to student and reservation modules.

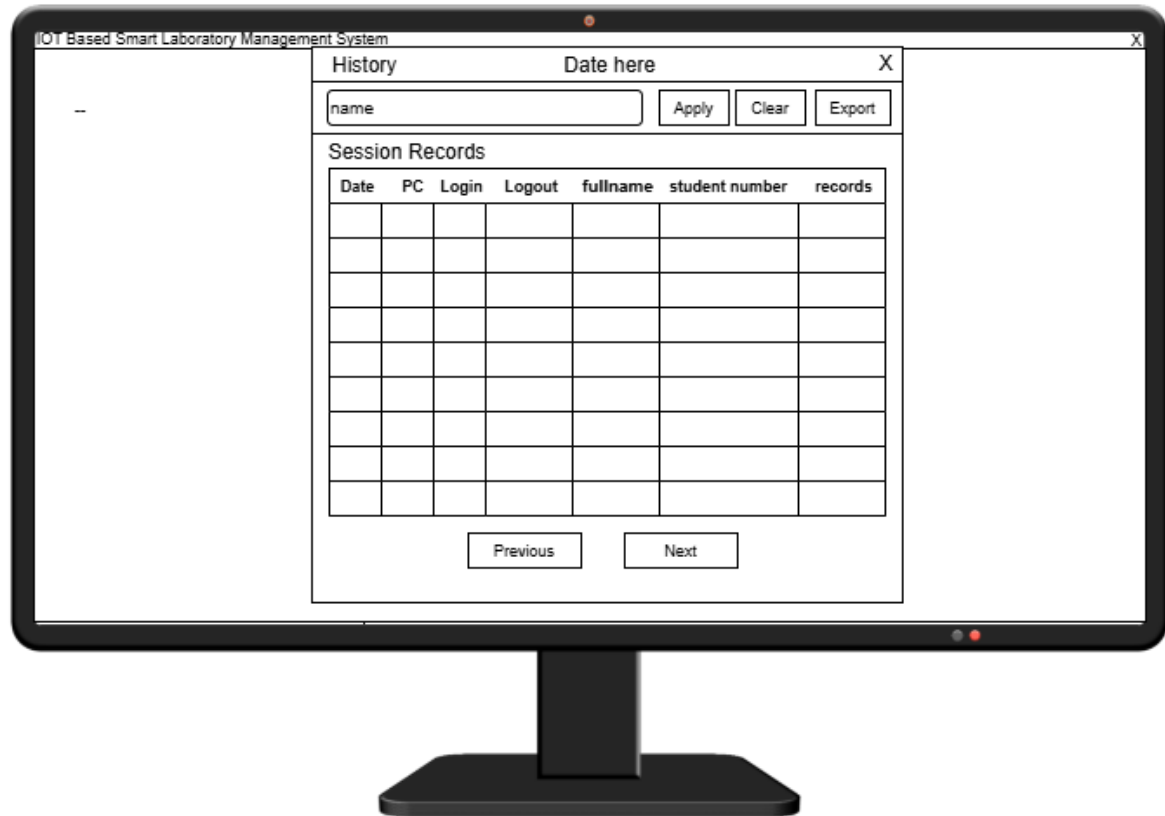


Figure 7. Session History Interface of the Admin Dashboard

The figure shows the session history interface, which displays recorded session data including login and logout time, workstation used, student details, and recorded files. It provides filtering, navigation, and export functions to support monitoring, tracking, and review of laboratory activities.



Figure 8. System Settings Interface of the Admin Dashboard

The figure shows the system settings interface, which allows configuration of streaming parameters, recording mode, interface themes, and session duration limits. It provides centralized control for adjusting system behavior and operational settings.

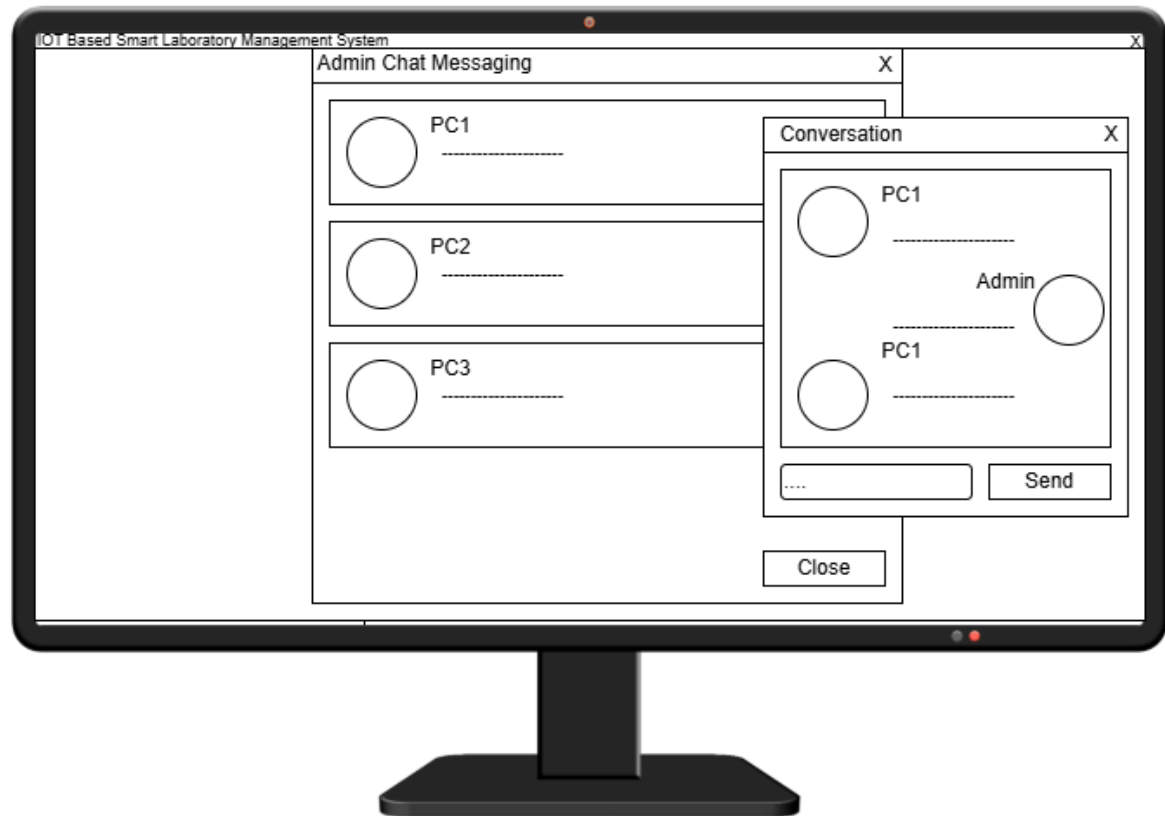


Figure 9. Admin Chat Messaging Interface of the System

The figure shows the admin chat messaging interface, which allows communication between the admin and selected workstation clients. It displays available PCs, active conversations, and a message input area for sending real-time messages to support coordination and supervision during laboratory sessions.



Figure 10. Reservation Management Interface of the Admin Dashboard

The figure shows the reservation management interface, which allows the admin to create, view, refresh, and cancel workstation reservations. It displays reservation details such as student number, assigned PC, start and end time, and status to support scheduling and resource allocation.

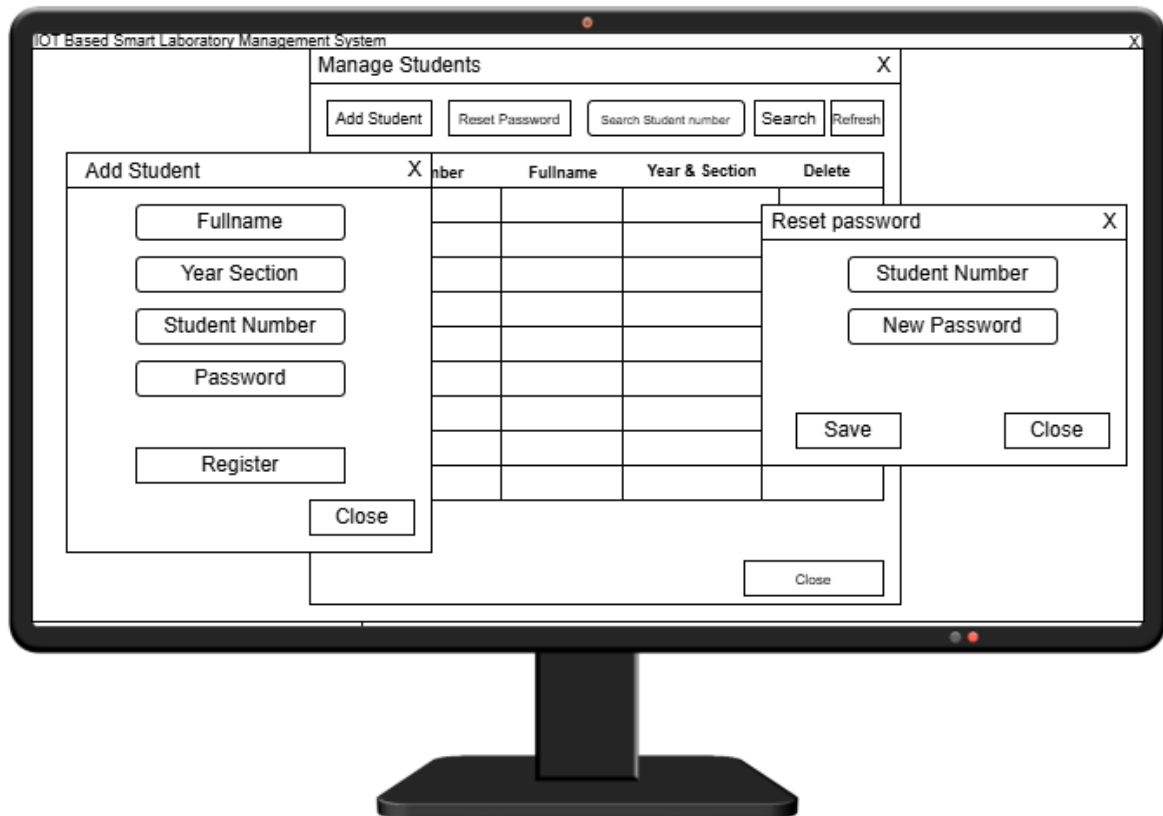


Figure 11. Student Management Interface of the Admin Dashboard

The figure shows the student management interface, which allows the admin to add, search, and manage student accounts. It includes functions for student registration, password reset, and record viewing to support user account administration within the system.



Use Case Diagram

The Use Case Diagram illustrates how external actors interact with the IoT-Based Smart Laboratory Management System. It identifies the Teacher (Admin) Operator, Student, and ESP32 Sensor Device and the services they perform within the system

Figure Definition

Symbols



Actors



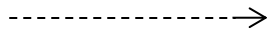
Association



Use Case



System Boundary



<<extend>>

Description

This represents a person or external entity that interacts with the system.

This represents the relationship between an actor and a use case.

This represents a function or service that the system performs.

This defines the scope of the system and separates it from external entities.

This represents a relationship where a use case optionally adds or extends the behavior of another use case under certain conditions.

Use Case Diagram

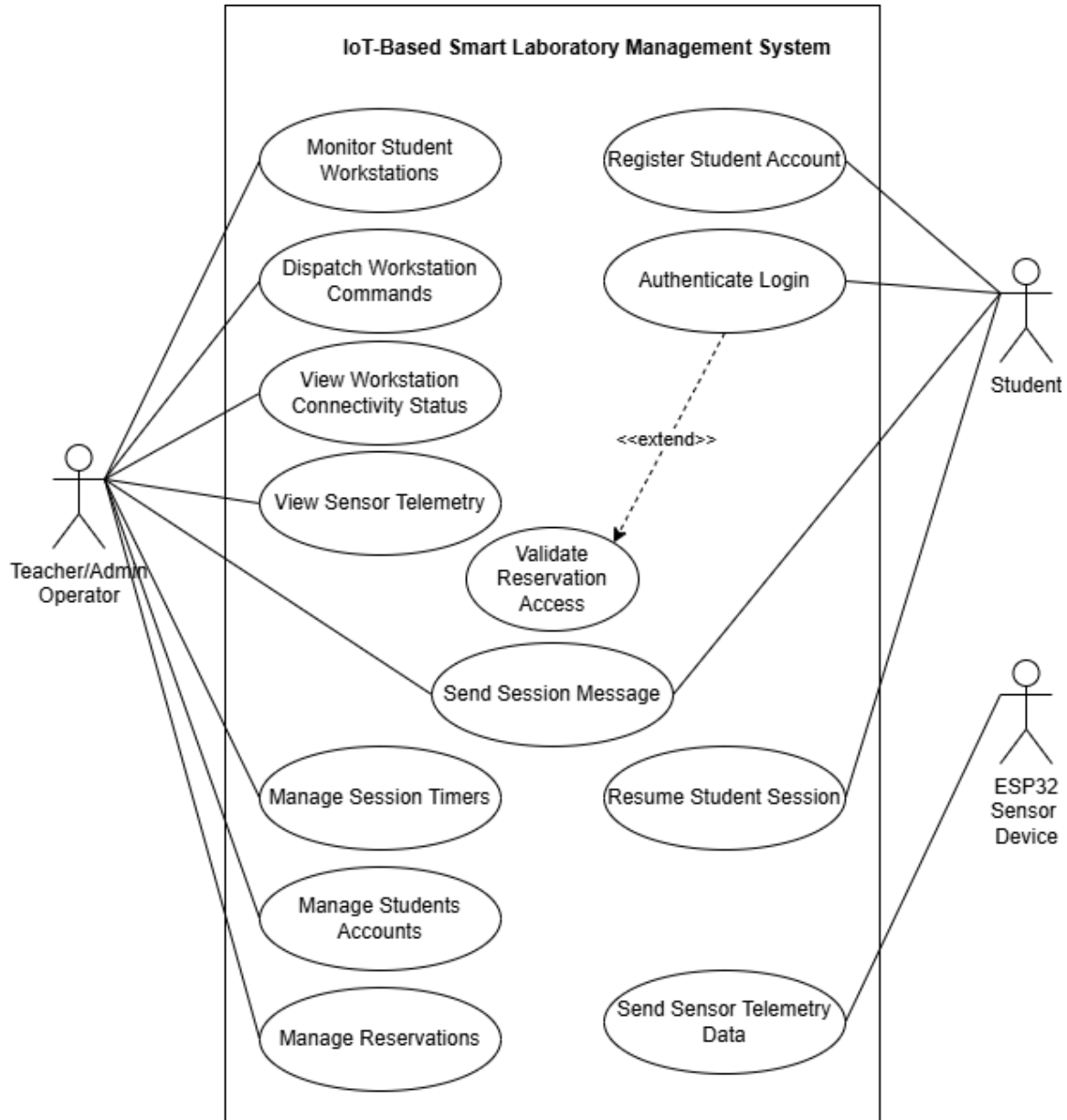


Figure 12. Use Case Diagram of the IoT-Based Smart Laboratory Management System

The figure illustrates the interaction between the Teacher (Admin) Operator, Student, and ESP32 Sensor Device with the system. It presents the main system functions including monitoring, control, authentication, session management, reservation handling, and sensor data transmission. The diagram also shows an <<extend>> relationship where reservation validation is conditionally performed during student login.



Process Design

The process design phase focuses on the designs of the elements within the system such as the architecture, modules and components, the different interfaces of those components and the data that goes through the system. The process design provides sufficient detailed data and information about the system, and its elements make's the implementation consistent with architectural entities as defined in models and views of the system architecture.

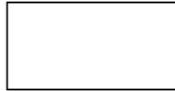
Data Flow Diagram

Data Flow Diagrams (DFDs) are used in this study to represent the logical flow of data within the proposed IoT-Based Smart Laboratory Management System. The diagrams illustrate how data moves between external entities and the system processes without revealing implementation details.

Figure Definition

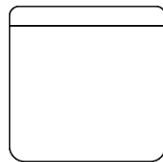
Symbols

Definition



Entity

Represents an external entity that interacts with the system by providing input or receiving output data.



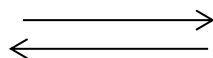
Process

Represents an activity or function that transforms input data into output within the system



Data Store

Represents a storage location where data is kept within the system.



Flow Line

Represents the movement of data between entities, processes, and data stores, indicating the direction of data flow.



Context Diagram (Level 0)

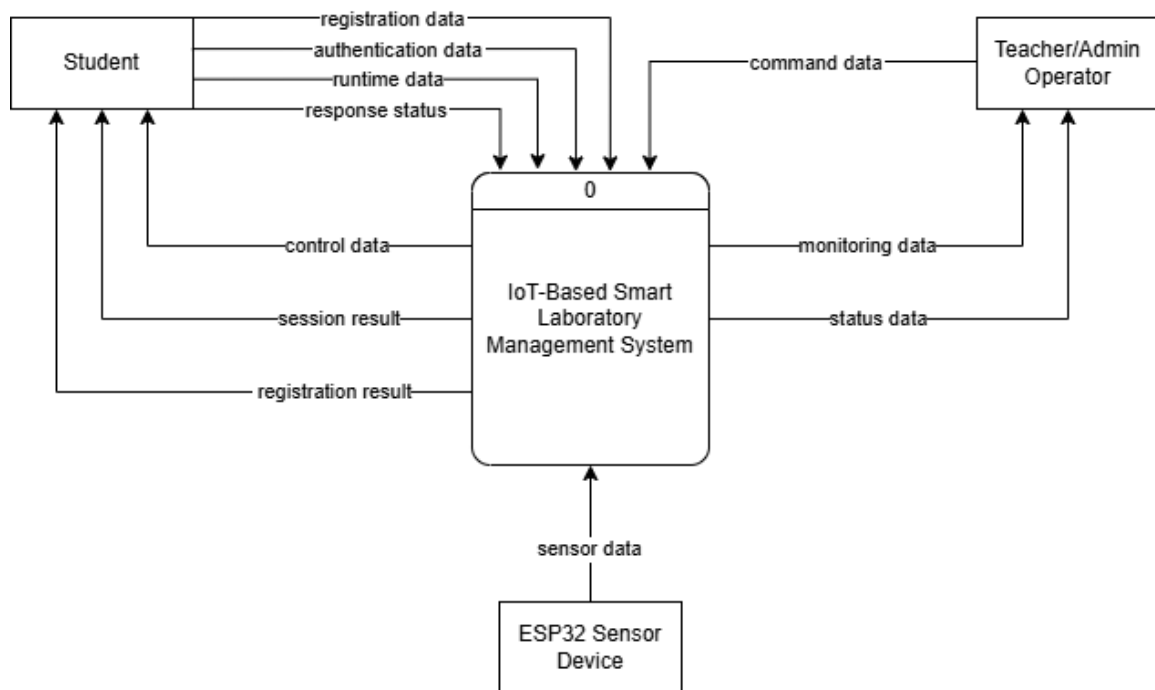


Figure 13. Context Diagram (Level 0 DFD)

This figure shows how the system interacts with users and external devices. It presents the overall system boundary and its interaction with the Student, Teacher (Admin) Operator, and ESP32 Sensor Device. It illustrates the high-level data flows including registration data, authentication data, runtime data, command data, monitoring data, status data, and sensor data exchanged between the system and external entities.



Level 1 Data Flow Diagram

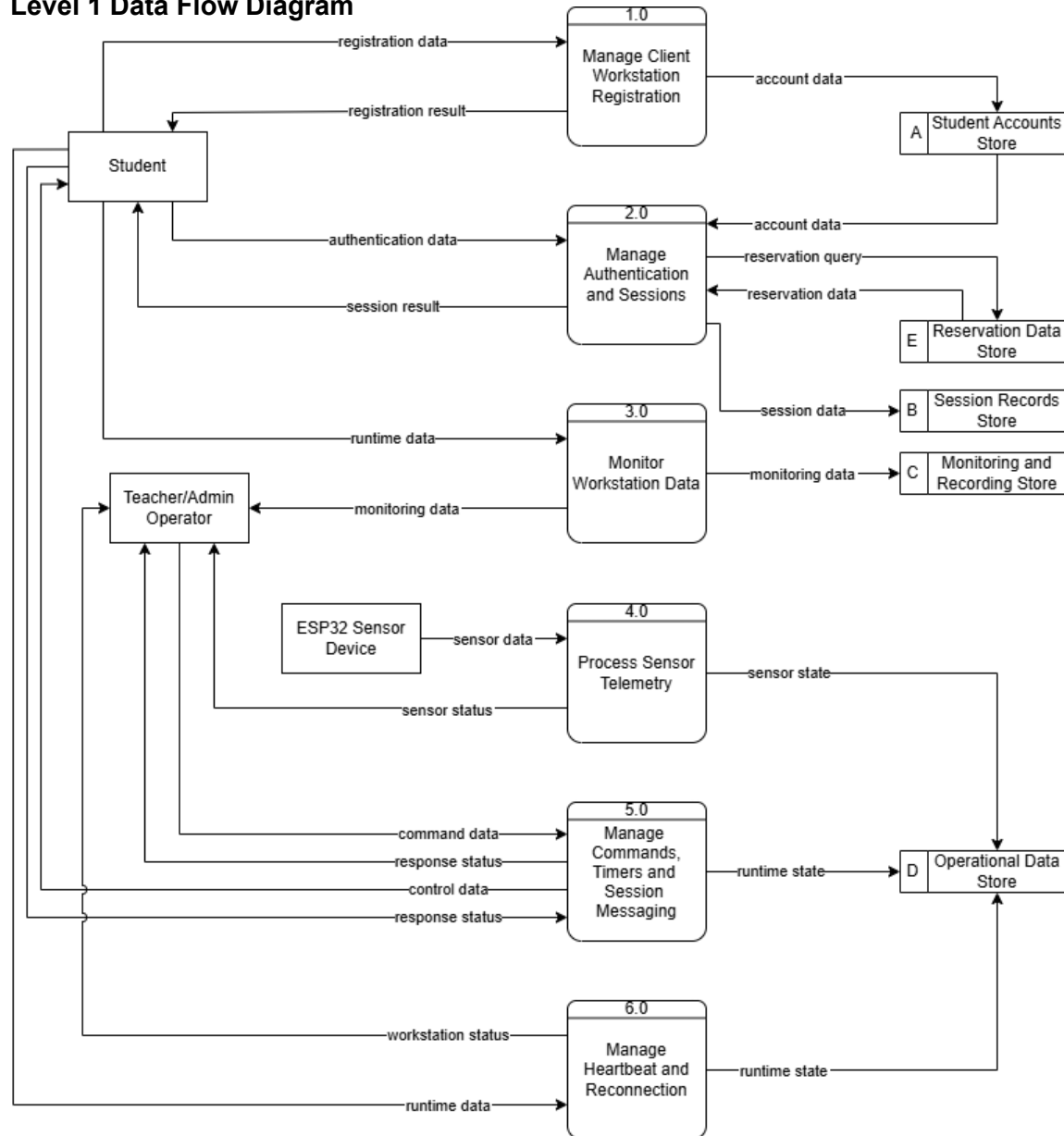


Figure 14. Data Flow Diagram (Level 1)

This figure shows the main functions of the system and how they work together. It presents the decomposition of the system into six processes: client workstation registration, authentication and session management, workstation monitoring, sensor telemetry processing, command and messaging management, and heartbeat and reconnection. It also shows how these processes interact with external entities and data stores such as the Student Accounts Store, Session Records Store, Monitoring and Recording Store, Operational Runtime Store, and Reservation Data Store.

Level 2 Decomposition

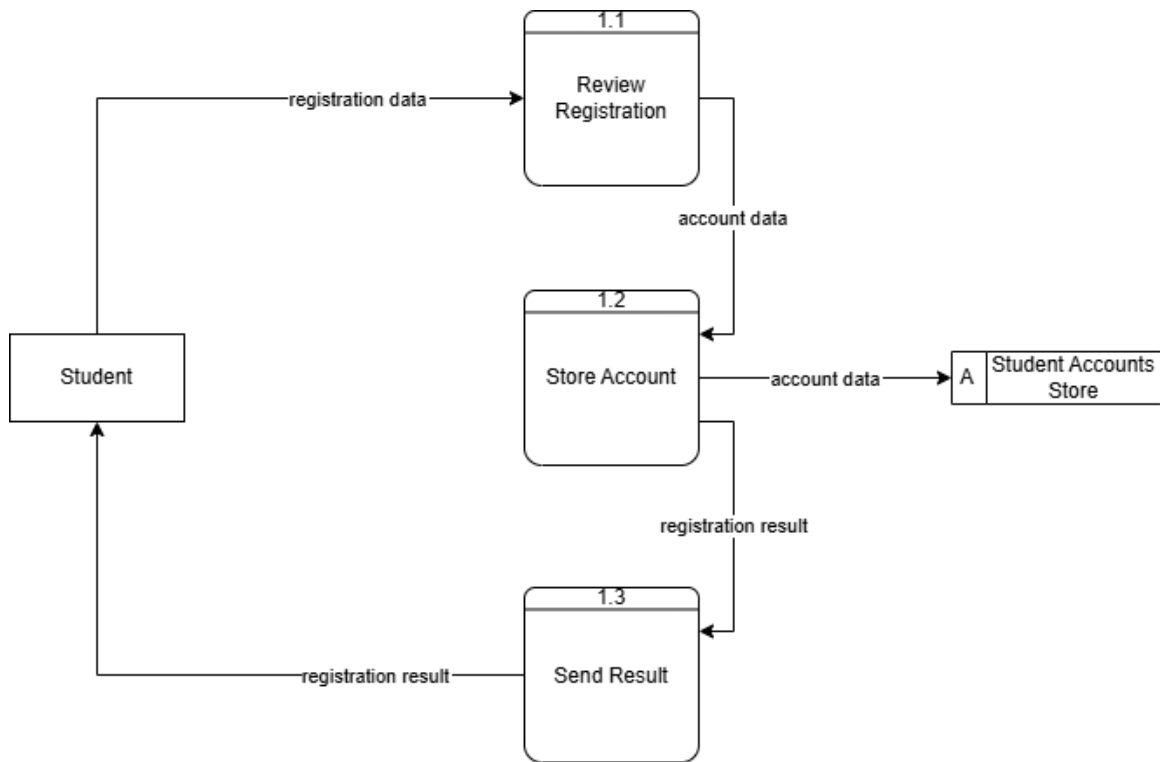


Figure 15. Level 2 Decomposition of 1.0 Manage Client Workstation Registration

This figure shows how a student registers a workstation in the system. It includes reviewing registration data, storing account data in the Student Accounts Store, and returning the registration result to the student.



Level 2 Decomposition

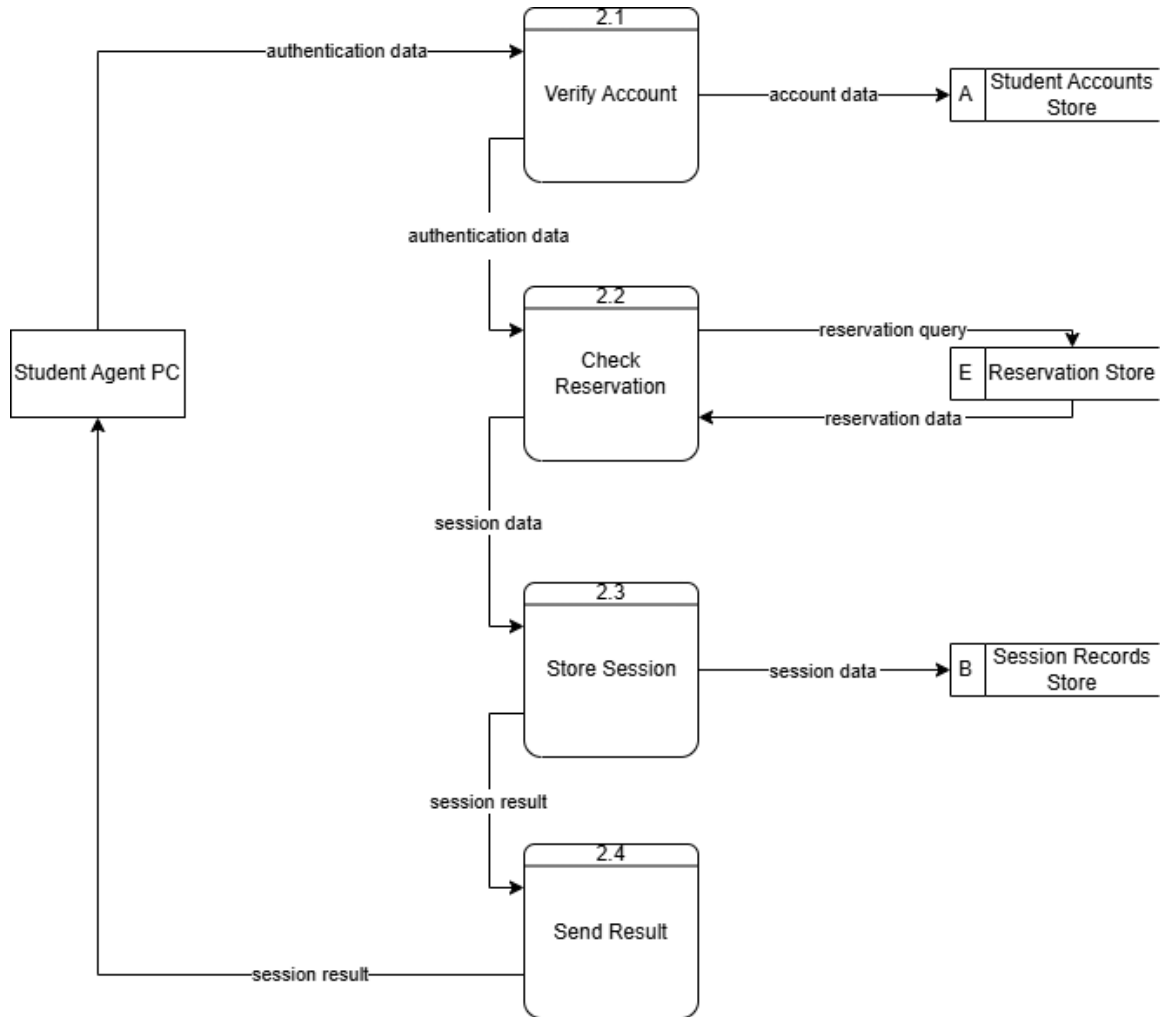


Figure 16. Level 2 Decomposition of 2.0 Manage Authentication and Sessions

This decomposition represents account validation, credential verification, session continuity, timeout/disconnection handling, and session-state publishing for downstream monitoring processes.



Level 2 Decomposition

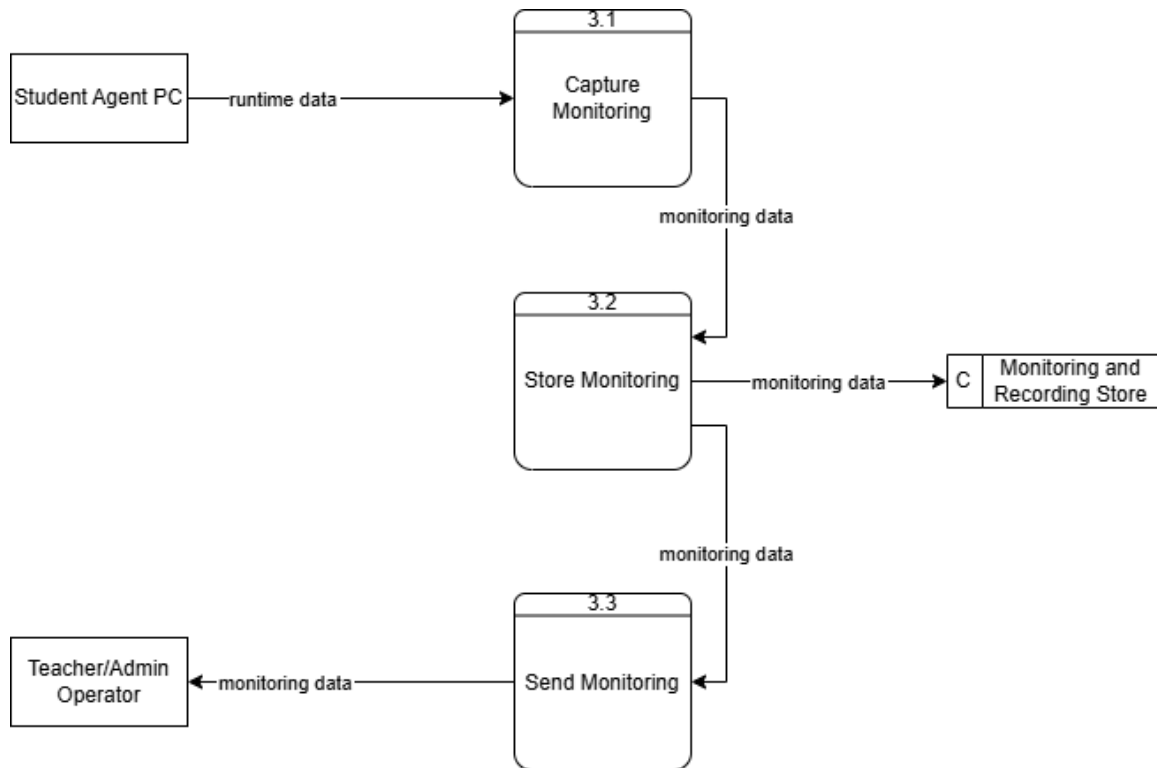


Figure 17. Level 2 Decomposition of 3.0 Monitor Workstation Data

This figure shows how the system monitors student workstation activity. It includes capturing runtime data, storing monitoring data in the Monitoring and Recording Store, and sending monitoring data to the administrator.

Level 2 Decomposition

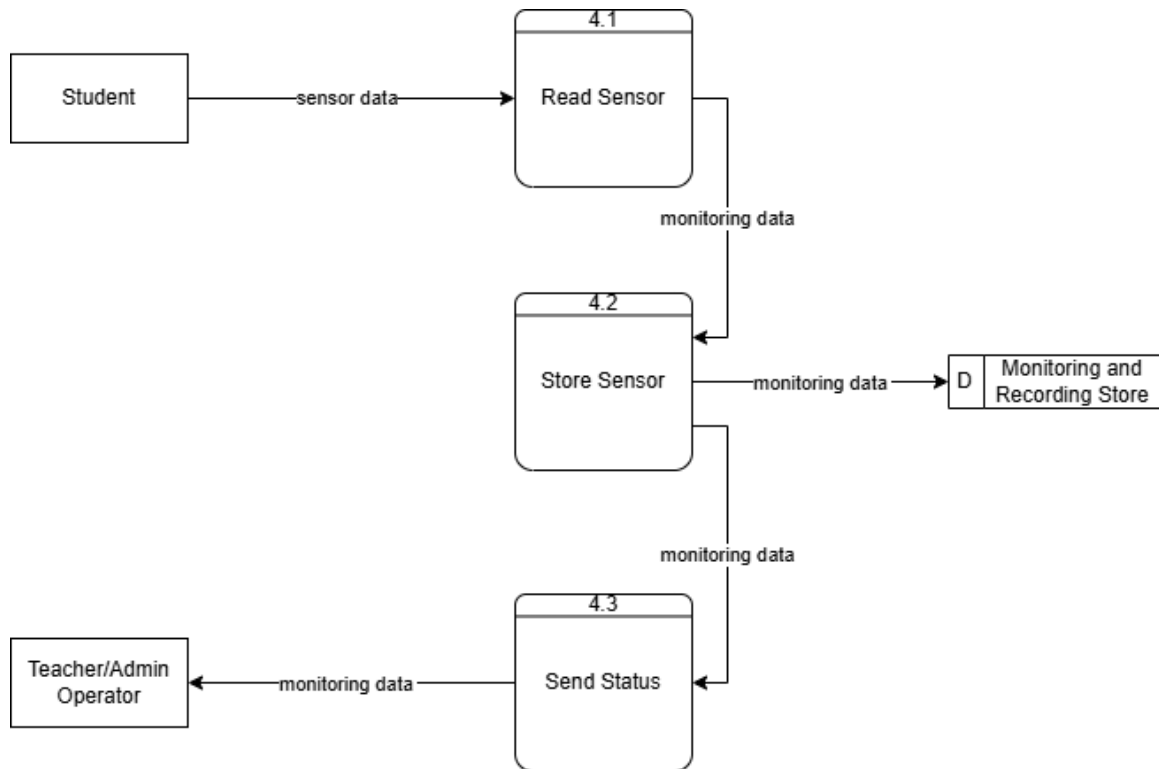


Figure 18. Level 2 Decomposition of 4.0 Process Sensor Telemetry

This figure shows how sensor data is collected and monitored. It includes processing sensor data from the ESP32 device, updating sensor state in the Operational Runtime Store, and sending sensor status to the administrator.



Level 2 Decomposition

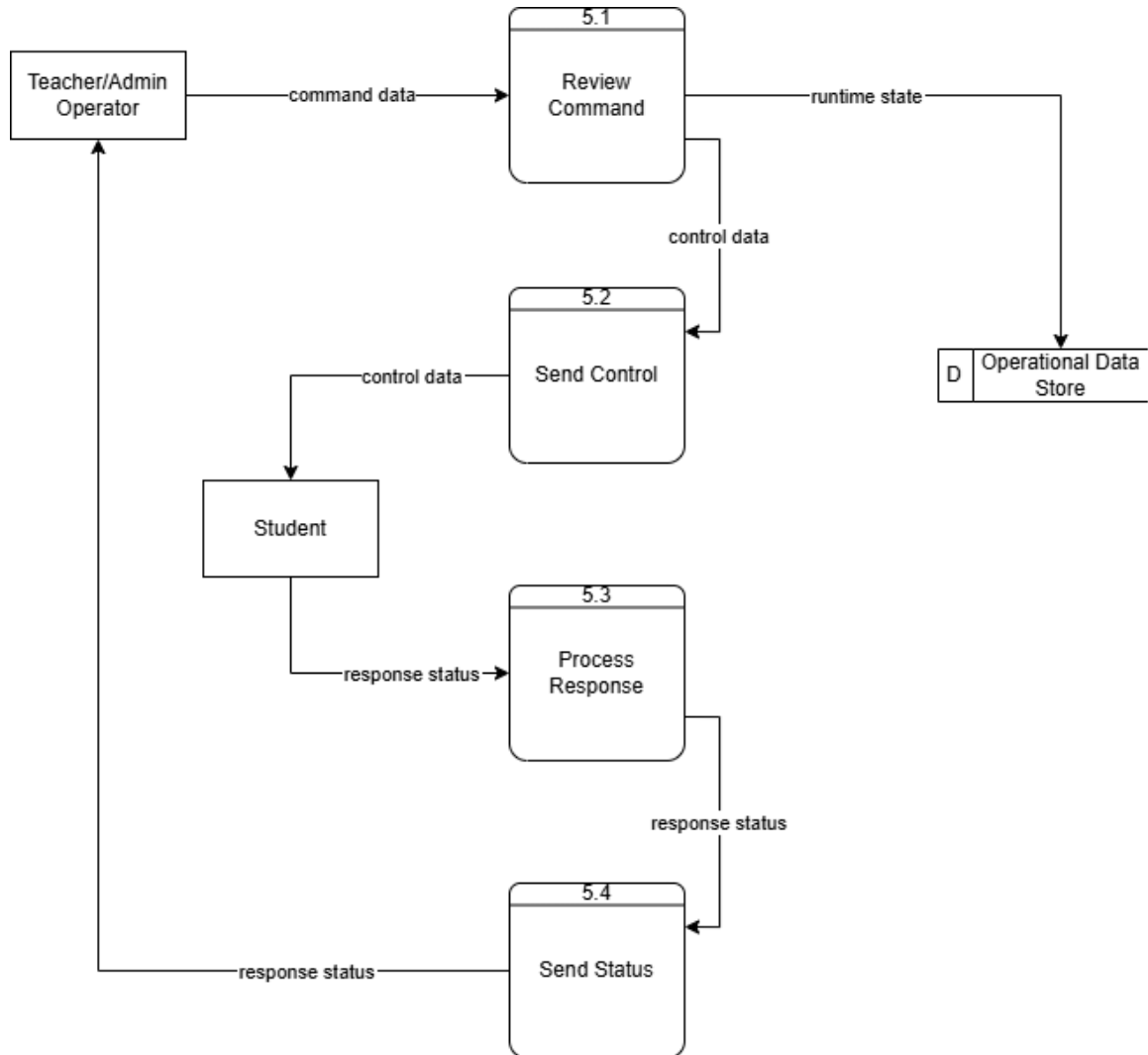


Figure 19. Level 2 Decomposition of 5.0 Manage Commands, Timers, and Session Messaging

This figure shows how the administrator sends commands to student workstations. It includes processing command data, updating runtime state, sending control data to the student, receiving response status, and returning response status to the administrator.

Level 2 Decomposition

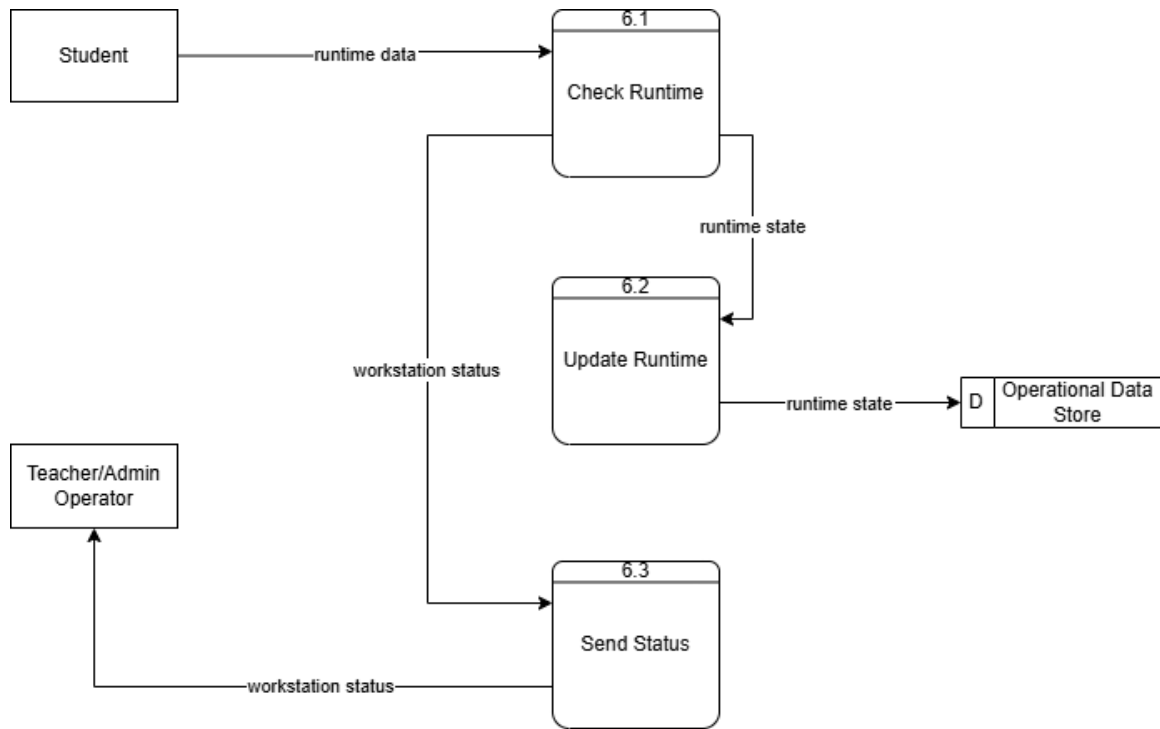


Figure 20. Level 2 Decomposition of 6.0 Manage Heartbeat and Reconnection

This figure shows how the system tracks workstation connectivity and status. It includes processing runtime data, updating runtime state in the Operational Runtime Store, and sending workstation status to the administrator.

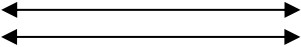
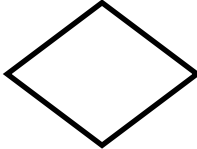
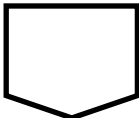


System Flowchart

This subsection presents the system flowcharts of the proposed system. System flowcharts are used to illustrate the logical sequence of operations and decision-making steps within major system functions.

Figure Definition

The following symbols are used in the system flowchart to represent the logical sequence of operations in the IoT-Based Smart Laboratory Management System.

Name	Symbol	Description
Terminal		Represents the starting point and ending point of the process flow.
Input/output		Represents data input into the system or output generated by the system.
Process		Represents a specific operation, action, or processing step performed by the system.
Flow lines		Indicates the direction and sequence of process flow within the system.
Decision		Represents a conditional statement or decision point that determines the flow path based on a given condition.
Display		Represents the presentation of information to the user through a visual interface.
On-page connector		Used to link separated parts of a flowchart within the same page, primarily to avoid long or intersecting flow lines and improve diagram readability.
Off- page connector		Used to indicate that the flow continues to another page, providing a reference point for the continuation of the process in a separate diagram.



SYSTEM FLOWCHART

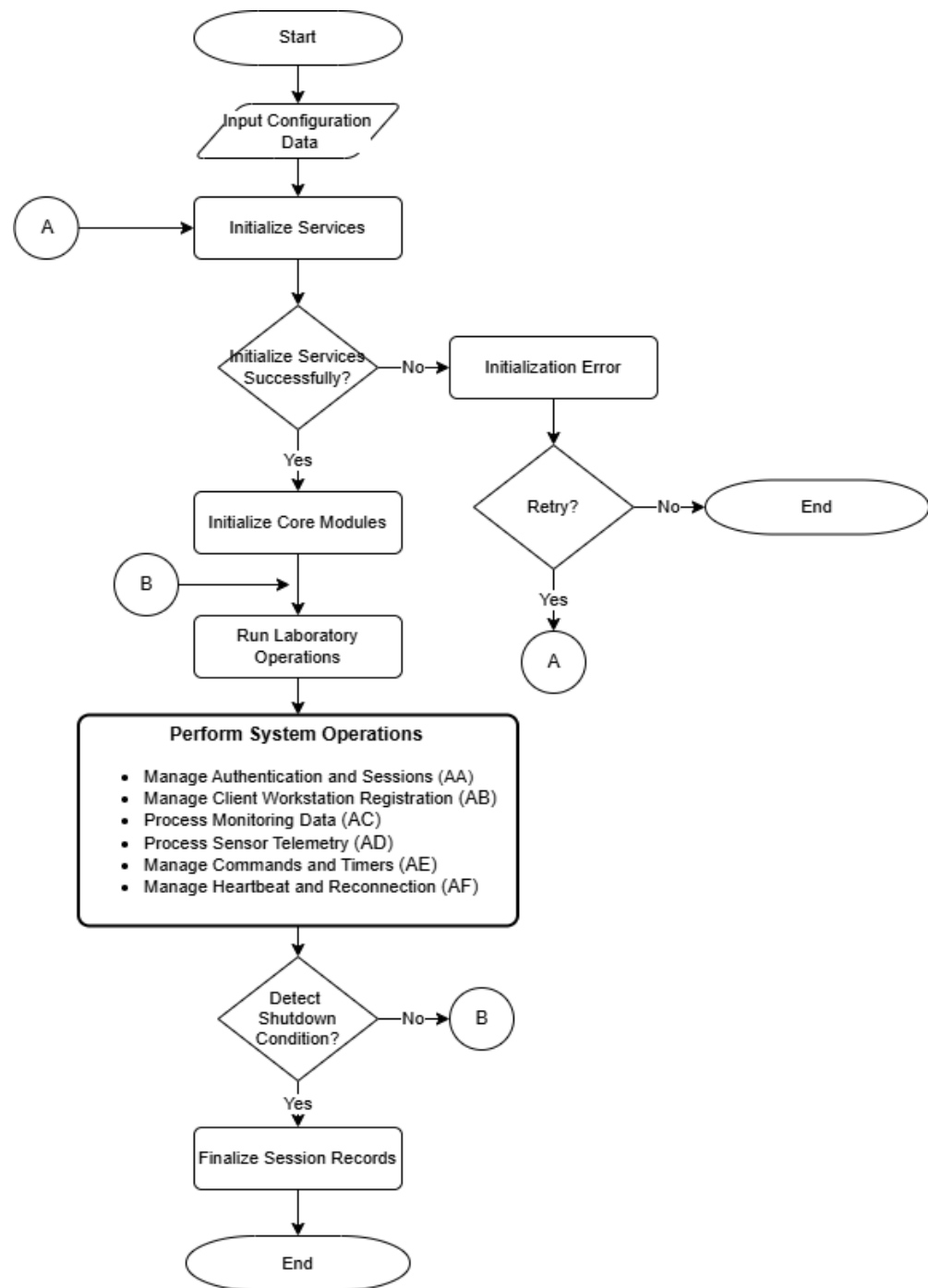


Figure 21. System Flowchart of the IoT-Based Smart Laboratory Management System

This flowchart illustrates the overall operation of the system, beginning with service initialization and progressing to the execution of core modules, including authentication, registration, monitoring, telemetry processing, command management, and connectivity



handling. The system continuously performs these operations until a shutdown condition is detected, after which session records are finalized before terminating the process.

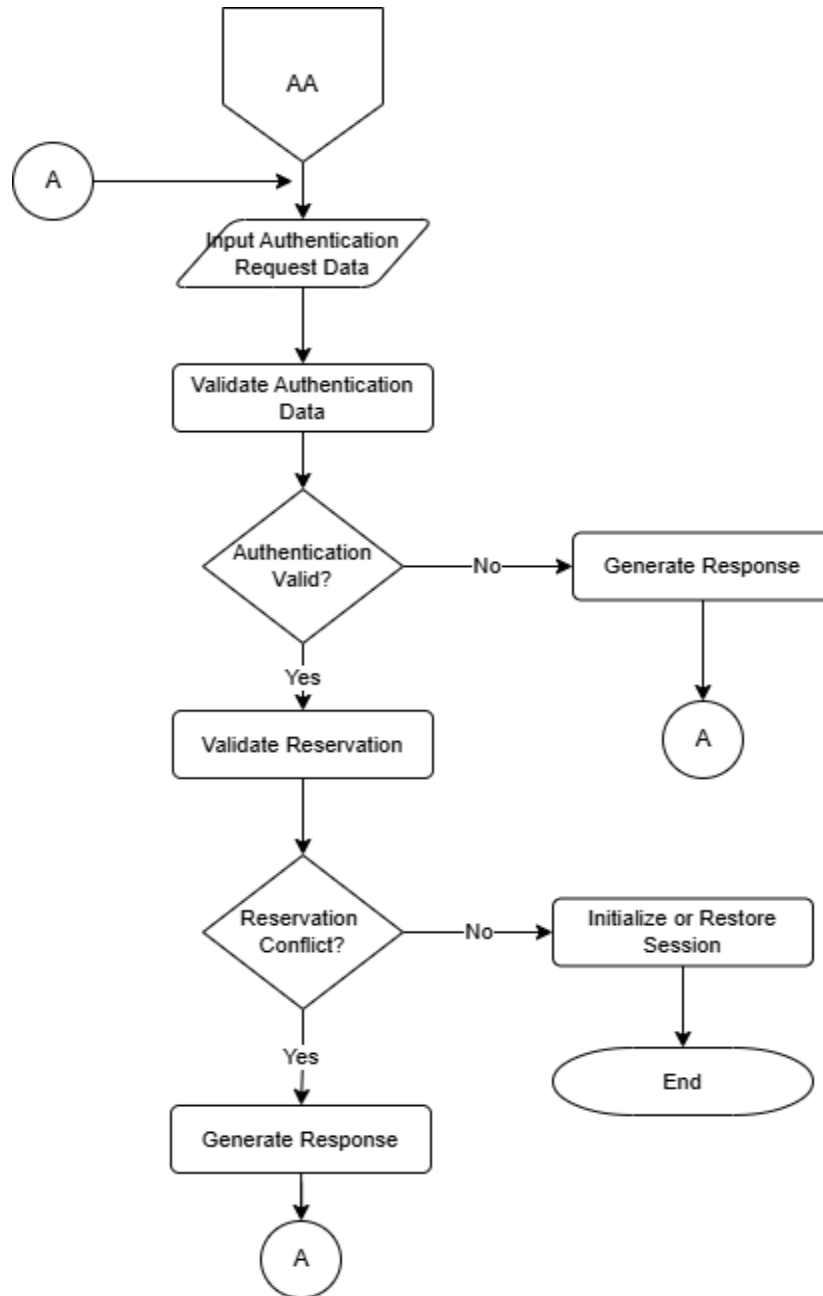


Figure 22. Module Flowchart for Manage Authentication and Sessions

This sub-process illustrates how the system controls user access by validating authentication data and verifying reservation status. If authentication fails or a reservation conflict is detected, the process is redirected for re-entry; otherwise, a session is initialized or restored to allow system access.

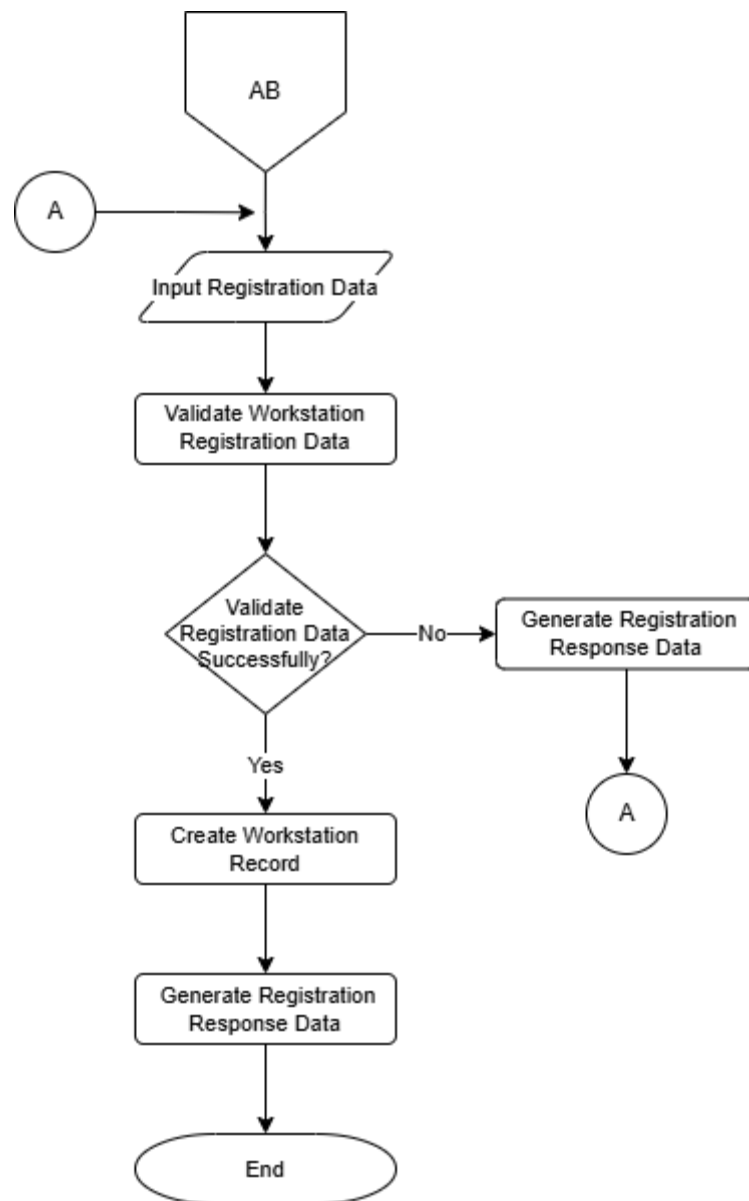


Figure 23. Module Flowchart for Manage Client Workstation Registration

This module illustrates the workstation registration process of the system, where registration data is received and validated to ensure correctness. Valid data results in the creation of a workstation record, while invalid data is redirected for re-entry to maintain accurate system registration.

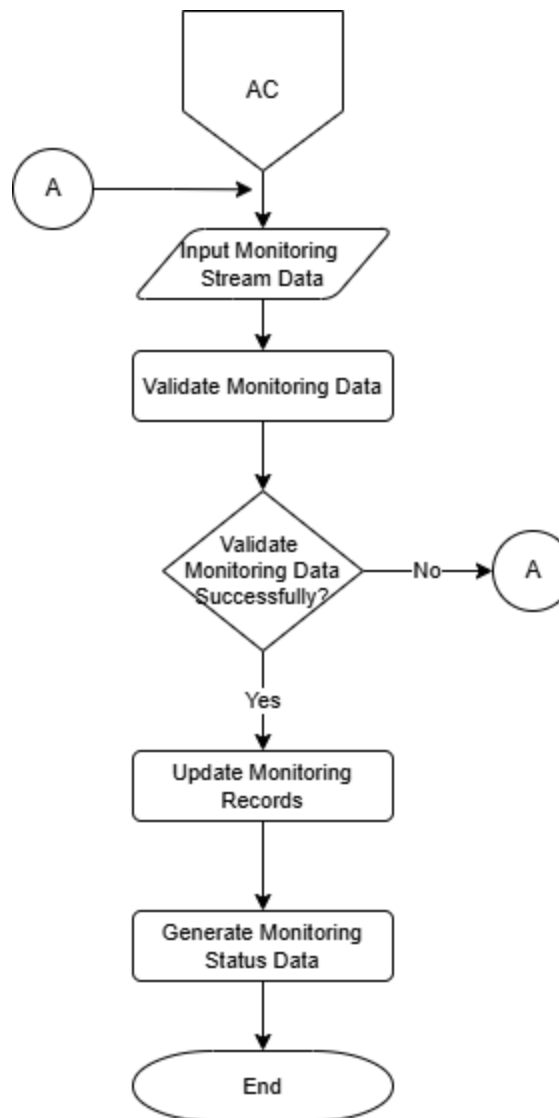


Figure 24. Module Flowchart for Process Monitoring Data

This sub-process illustrates how the system processes monitoring stream data by receiving and validating incoming information to ensure accuracy. Valid data is used to update monitoring records and generate status information for system observation, while invalid data is redirected for re-entry.

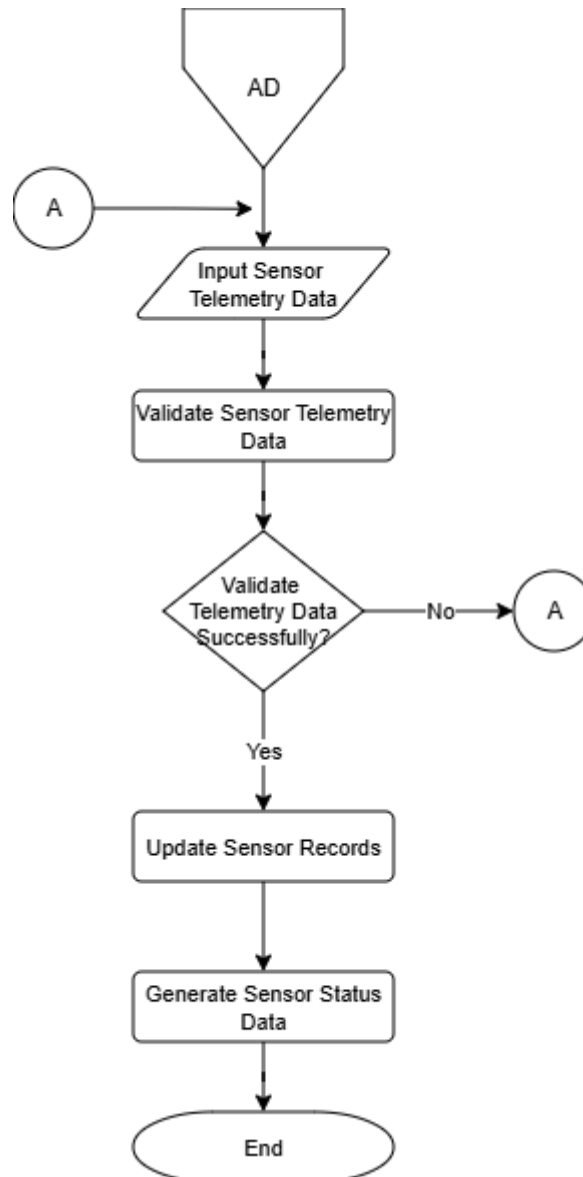


Figure 25. Flowchart Module Flowchart for Process Sensor Telemetry

This sub-process illustrates how the system handles sensor telemetry by receiving and validating incoming data to ensure accuracy. Valid data is used to update sensor records and generate status information for monitoring, while invalid data is redirected for re-entry.

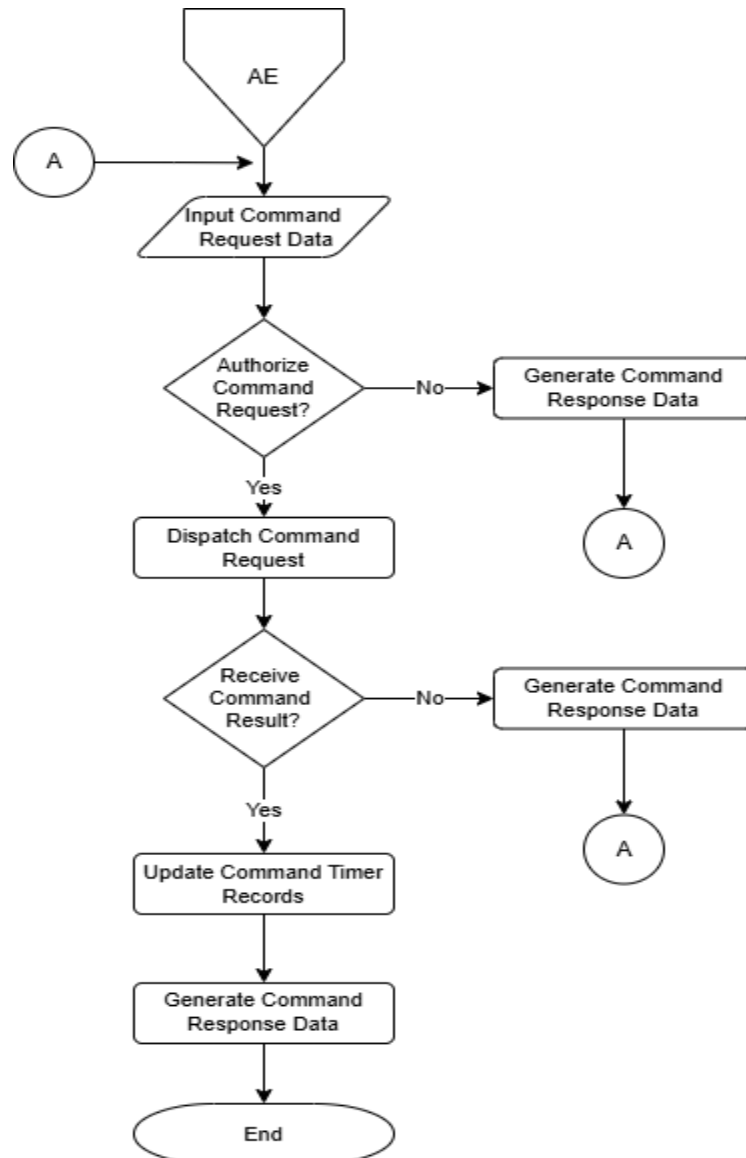


Figure 26. Module Flowchart for Manage Commands, Timers, and Session Messaging

This sub-process illustrates how the system processes command requests by verifying authorization before execution. Authorized commands are transmitted to the corresponding workstation, and the resulting output is evaluated to update command timer records. If the request is not authorized or no valid result is received, the process is redirected for re-entry.

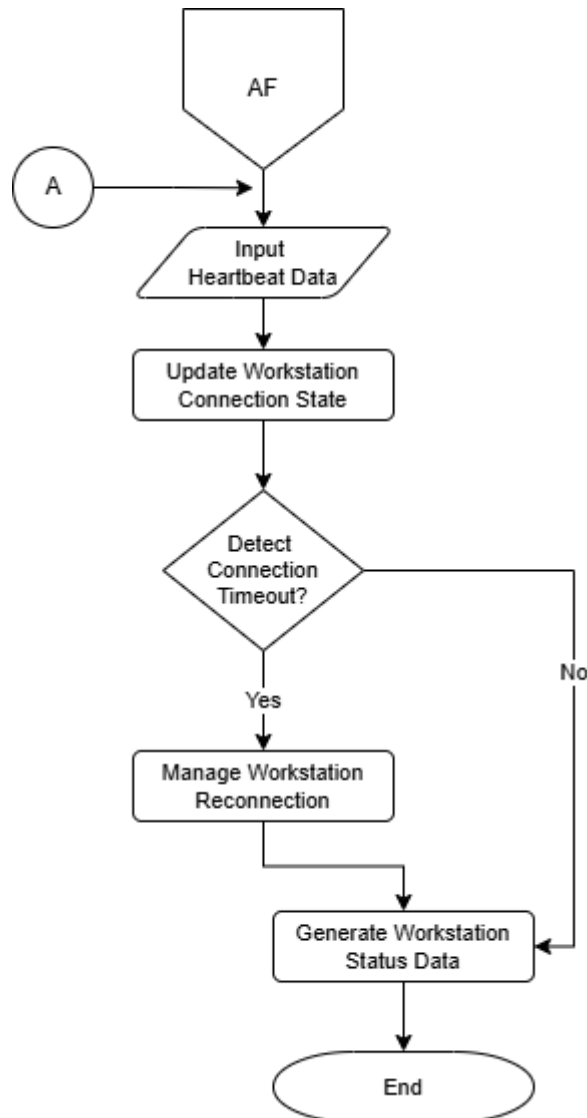


Figure 27. Module Flowchart for Manage Heartbeat and Reconnection

This sub-process illustrates how the system monitors workstation connectivity by processing heartbeat data and updating the connection status. The system detects connection timeouts and performs reconnection procedures when necessary, ensuring that each workstation maintains a stable and accurate connection state.

ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) is a visual representation of the logical structure of a database that illustrates entities, their attributes, and the relationships between them. It is commonly used in system design to model how data is stored, organized, and related within a system.

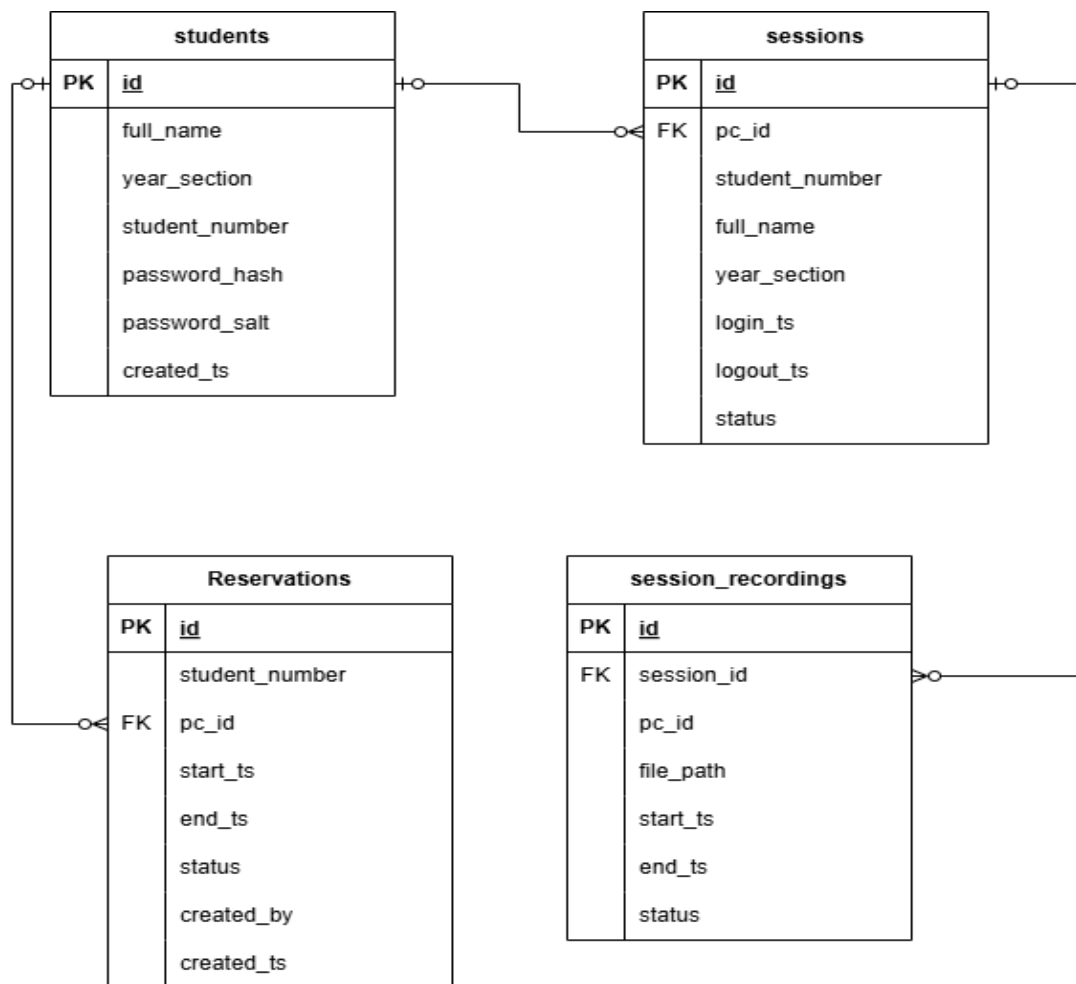


Figure 28. Entity-Relationship Diagram of the IoT-Based Smart Laboratory Management System

The Entity Relationship Diagram (ERD) presents four main entities: Students, Sessions, Reservations, and Session Recordings. It shows one-to-many relationships where a student can have multiple sessions and reservations, and each session can have multiple recordings. The diagram follows a centralized structure using Crow's Foot notation.



DATA DICTIONARY

The data dictionary defines the field structure of the persistent relational tables used by the system database. Each entry specifies the field name, data type, constraint conditions, and descriptive purpose of the stored attribute. This specification ensures consistent interpretation of database schema elements and supports maintainability of the persistence layer.

Table 5. Students

Field Name	Data Type	Constraint	Description
id	INTEGER	PK, Auto Increment	Internal student identifier
full_name	TEXT	NOT NULL	Student full name
year_section	TEXT	NOT NULL	Year and section
student_number	TEXT	UNIQUE, NOT NULL	Student identity/login key
password_hash	TEXT	NOT NULL	Salted password hash
password_salt	TEXT	NOT NULL	Password salt
created_ts	REAL	NOT NULL	Registration timestamp

The students table stores the registered user accounts of student users in the system. It contains essential identity information, authentication credentials, and registration metadata required for login and user management. This table serves as the primary reference for student-related operations such as authentication, session tracking, and reservation linkage.

Table 6. Sessions

Field Name	Data Type	Constraint	Description
id	INTEGER	PK, Auto Increment	Session identifier
pc_id	TEXT	NOT NULL	Workstation identifier
student_number	TEXT	NOT NULL	Logical link to student identity
full_name	TEXT	NOT NULL	Student name snapshot
year_section	TEXT	NOT NULL	Section snapshot
login_ts	REAL	NOT NULL	Login timestamp



logout_ts	REAL	Nullable	Logout timestamp
status	TEXT	NOT NULL	Session state

The Sessions table records all active and past login sessions of students within the laboratory system. It captures session-related data such as workstation usage, login and logout times, and session status. This table enables monitoring of user activity and supports session management within the system.

Table 7. Session Recordings

Field Name	Data Type	Constraint	Description
Id	INTEGER	PK, Auto Increment	Recording row identifier
session_id	INTEGER	FK sessions.id, NOT NULL	Linked session identifier
pc_id	TEXT	NOT NULL	Workstation identifier
file_path	TEXT	NOT NULL	Recording file path
start_ts	REAL	NOT NULL	Recording start timestamp
end_ts	REAL	Nullable	Recording end timestamp
status	TEXT	NOT NULL	Recording status

The Session Recordings table stores metadata of recorded session activities within the system. It links each recording to a specific session and workstation, including file location and recording duration. This table supports playback, auditing, and review of recorded laboratory sessions.

Table 8. Reservations

Field Name	Data Type	Constraint	Description
id	INTEGER	PK, Auto Increment	Reservation identifier
student_number	TEXT	FK students.student_number, NOT NULL	Student identity reference
pc_id	TEXT	NOT NULL	Workstation identifier



start_ts	REAL	NOT NULL	Reservation start timestamp
end_ts	REAL	NOT NULL	Reservation end timestamp
status	TEXT	NOT NULL	Reservation state
created_by	TEXT	NOT NULL	Reservation creator
created_ts	REAL	NOT NULL	Reservation creation timestamp

The Reservations table manages scheduled workstation usage within the laboratory. It records reservation details such as assigned student, workstation, time duration, and reservation status. This table supports session planning, scheduling, and conflict management within the system.



SYSTEM BLOCK DIAGRAM

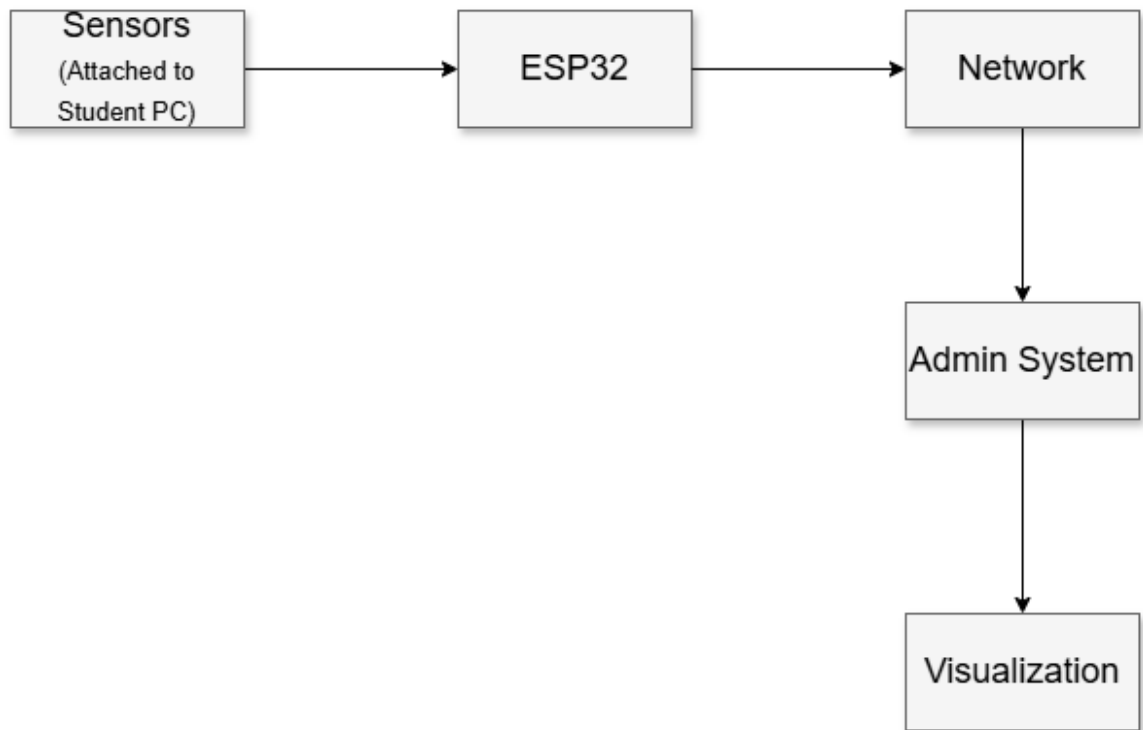


Figure 29. System Block Diagram of the IoT-Based Smart Laboratory Management System

This figure presents the system block diagram of the monitoring system, showing the flow of data between its major components. Sensors attached to the student workstation collect hardware data, such as temperature and fan activity, and transmit these readings to the ESP32 microcontroller for initial processing. The ESP32, operating in station mode, sends the processed sensor data through the local network to the admin system. The admin system then performs data handling and analysis, and the results are displayed through the visualization interface for real-time monitoring and supervision.

SYSTEM ARCHITECTURE

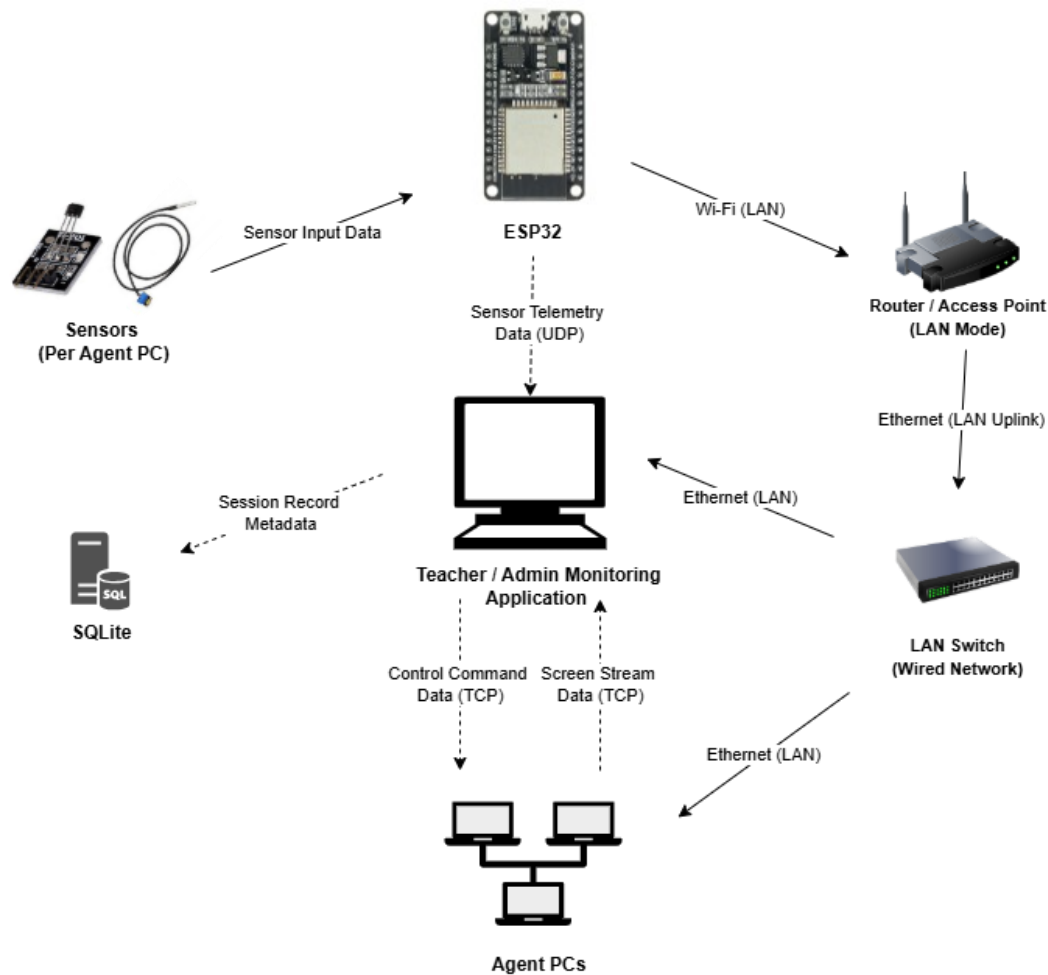


Figure 30. System Architecture (centralized multi-channel client-server architecture over a local area network)

Figure 30 presents the system architecture of the IoT-Based Smart Laboratory Management System developed for the ESSU Arteche campus. The architecture describes the logical and runtime arrangement of system components, their roles, and their communication relationships within the laboratory Local Area Network (LAN) environment. The implemented architecture follows a centralized client-server model, where the Teacher / Admin Monitoring Application (Python) directly manages all connected Students and sensor nodes. The architecture is organized into three functional layers: the Monitoring and Control Layer, the Client and Communication Layer, and the Sensor Layer.



The Monitoring and Control Layer consists of the Teacher / Admin Monitoring Application with its graphical user interface. This component serves as the central authority for authentication, session management, command issuance, video stream reception, telemetry processing, and local data storage. The Teacher / Admin Monitoring Application hosts multiple network channels: separate TCP channels for control commands and for receiving live screen stream data, and UDP channels for receiving sensor telemetry and heartbeat signals. The server operates entirely within the laboratory's local area network and does not function as a cloud-based or remote-hosted service.

The Client and Communication Layer include the student applications deployed on laboratory workstations. Each Student application on laboratory workstations establishes TCP channels to the Teacher / Admin Monitoring Application for control command exchange and screen streaming; UDP is used for heartbeat signaling and for workstation telemetry packets. The Teacher process directly manages these socket connections without the use of any external message broker, publish-subscribe middleware, or cloud intermediary. All session coordination, command acknowledgment handling, and connection monitoring are performed by the centralized server process. Command acknowledgments and session lifecycle events are exchanged over the TCP control channel and persisted as session record metadata; recording files and related metadata are saved locally on the Teacher host.

The Sensor Layer consists of the ESP32-based sensor module installed at the workstation. The ESP32 microcontroller collects environmental and workstation telemetry and transmits this telemetry to the Teacher / Admin Monitoring Application using UDP packets over the LAN. The sensor node operates as an independent telemetry source but remains logically integrated into the centralized architecture.

All components communicate within a trusted local area network consistent with the defined scope and limitations of the study. Persistent data including registered user accounts, session logs and recording files are stored locally on the Teacher host using SQLite databases and JSON files, in accordance with the implemented system. The architecture emphasizes centralized control, real-time monitoring, multi-channel LAN communication, and local deployment; it does not employ external cloud services, third-party message brokers, or enterprise-level distributed infrastructure.

Schematic Diagram

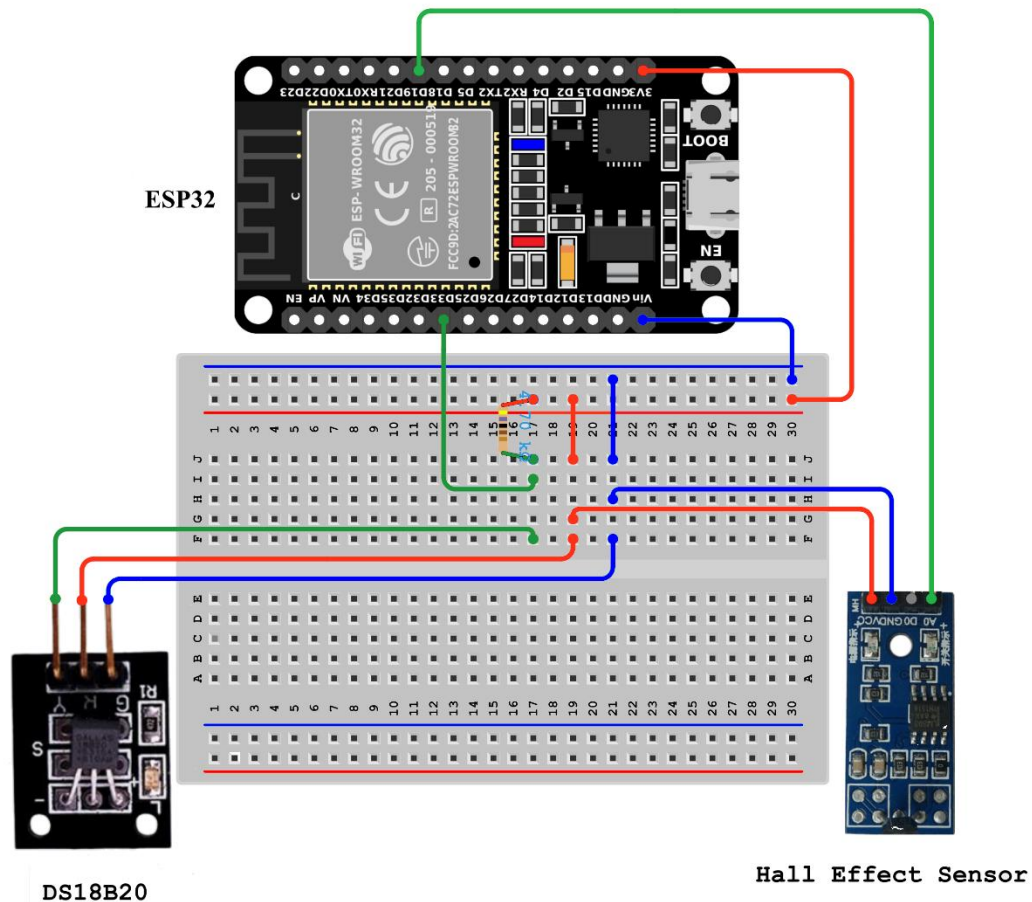


Figure 31. ESP32 Sensor Connection Diagram

The figure shows the connection of the ESP32 with a DS18B20 temperature sensor and a Hall effect sensor using a breadboard setup. The DS18B20 is placed on the CPU heatsink to measure its temperature, providing an indication of the system's thermal condition, and communicates with the ESP32 using the One-Wire protocol with a 4.7 kΩ pull-up resistor. The Hall effect sensor detects fan rotation to determine RPM through pulse signals. Both sensors are connected to the ESP32 via GPIO pins and powered through the breadboard, enabling real-time monitoring of temperature and cooling performance within the system.



Development

This section presents the planned development structure of the proposed IoT-Based Smart Laboratory Management System for ESSU Arteche Campus. It explains how the approved system designs are intended to be translated into structured program components, consistent with the adopted Prototyping-driven development model. Emphasis is placed on modular program organization, clearly defined responsibilities, and controlled integration of software and hardware components.

This section defines the intended program specifications, input and output descriptions, runtime data structures, operational logic, and programming environment that guide system construction. It does not present deployment outcomes or performance results.

Program Specifications

This subsection defines the planned major software components of the system: the Admin Monitoring Application, the student PC Monitoring Program, and the ESP32 Sensor Program.

The Admin Monitoring Application serves as the central monitoring and control unit. It is designed to receive runtime monitoring and telemetry data, process information, issue workstation control commands, and present real-time system status through the graphical interface. Communication is coordinated through direct TCP and UDP socket channels within the local area network.

The student PC Monitoring Program operates on each monitored workstation. It captures screen activity, responds to administrative commands, and transmits monitoring data during active laboratory sessions. The program runs in the background and supports session-based supervision.

The ESP32 Sensor Program manages sensor data acquisition and telemetry preparation. It reads sensor inputs, preprocesses values, and publishes telemetry data to the central system. It operates independently and focuses solely on sensor-related tasks.

Each component is designed as a modular unit to support independent development and controlled integration.

Input Descriptions

Inputs originate from the Admin Monitoring Application, student PC Monitoring Program, and ESP32 Sensor Program.



Administrative inputs include workstation selection commands, session controls, and monitoring configuration settings. student inputs consist of real-time screen data and workstation status information. Sensor inputs include telemetry readings acquired and preprocessed by the ESP32 module.

All inputs are temporary runtime data processed in real time. The system does not utilize user-submitted files or historical records; however, local persistent storage mechanisms are used for user authentication, session management, configuration data, and runtime operational records.

Output Descriptions

Outputs are generated during active monitoring sessions and are primarily displayed through the Admin Monitoring Application. These include real-time screen previews, system status indicators, session timing information, and sensor telemetry readings.

Screen outputs provide visual supervision of monitored workstations, while session outputs reflect control states and timing information. Sensor outputs display contextual workstation health data.

All outputs are dynamic runtime visual displays. The system primarily generates real-time outputs; however, selected operational data such as session logs and recordings are stored locally for monitoring and review purposes.

Data Structure Descriptions

The system utilizes temporary in-memory data structures to support real-time monitoring and control, together with local persistent storage mechanisms for user, session, configuration, and runtime management data.

The workstation data structure stores identifiers, connection status, lock state, session timers, and screen references. The screen monitoring structure temporarily holds incoming screen capture data. The sensor telemetry structure maintains current sensor readings and workstation associations. The command structure represents administrative instructions, including command type, target workstation, and execution state. These structures guide runtime data handling and do not represent database schemas.

Program Descriptions

The Admin Monitoring Application initializes network communication, processes incoming monitoring and telemetry data, applies administrative control logic, and updates the graphical interface.



The student PC Monitoring Program captures screen activity, transmits monitoring data, enforces received control commands during sessions, and participates in authentication and session-continuity handling through the implemented login and session-resume workflow.

The ESP32 Sensor Program continuously reads sensor inputs, preprocesses values, and publishes telemetry data through the defined communication mechanism. These descriptions clarify component responsibilities without presenting source code or performance outcomes.

Programming Environment

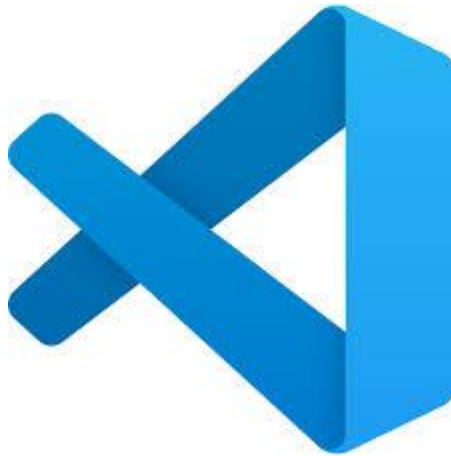


Figure 32. Programming Environment Visual Studio Code

Visual Studio Code is planned as the primary development environment for implementing the system software. It will support Python-based development, code editing, debugging, and modular programming required for the LAN-based monitoring system. The integrated terminal, extension marketplace, and built-in Git support are intended to facilitate efficient development of the Admin Monitoring Application and student PC program. Features such as IntelliSense, syntax highlighting, and debugging tools will enable the researchers to write, test, and maintain clean, functional code throughout the prototyping process.



Figure 33. Programming Environment Arduino IDE

Arduino IDE is planned for developing and uploading firmware to the ESP32 sensor device. The environment is intended to support writing, compiling, and deploying embedded programs required for sensor telemetry transmission. Its extensive library manager will provide access to pre-built drivers for the DS18B20 temperature sensor and Hall effect sensor, simplifying hardware interfacing. The serial monitor tool within the IDE will allow real-time monitoring of sensor output and debugging of the ESP32 module during development and testing phases.

Evaluation

The proposed IoT-Based Smart Laboratory Management System for ESSU Arteche Campus will be evaluated to determine whether it meets its stated functional and operational objectives. The assessment will follow the specified ISO/IEC 25010 software quality standards to ensure a structured and standards-based evaluation appropriate for an academic prototype system within the defined laboratory environment.

Research Design

This study will employ a descriptive-developmental research design. The descriptive aspect will document system performance and quality characteristics during testing, while the developmental aspect will assess whether the developed system satisfies the intended laboratory supervision requirements. This approach is appropriate for evaluating a prototype system within a controlled academic setting.

Locale

The evaluation will be conducted at Eastern Samar State University Arteche Campus, specifically within the College of Computer Studies laboratory. Testing will take place in the campus local area network (LAN) environment where instructor and student workstations are configured for supervised laboratory sessions.

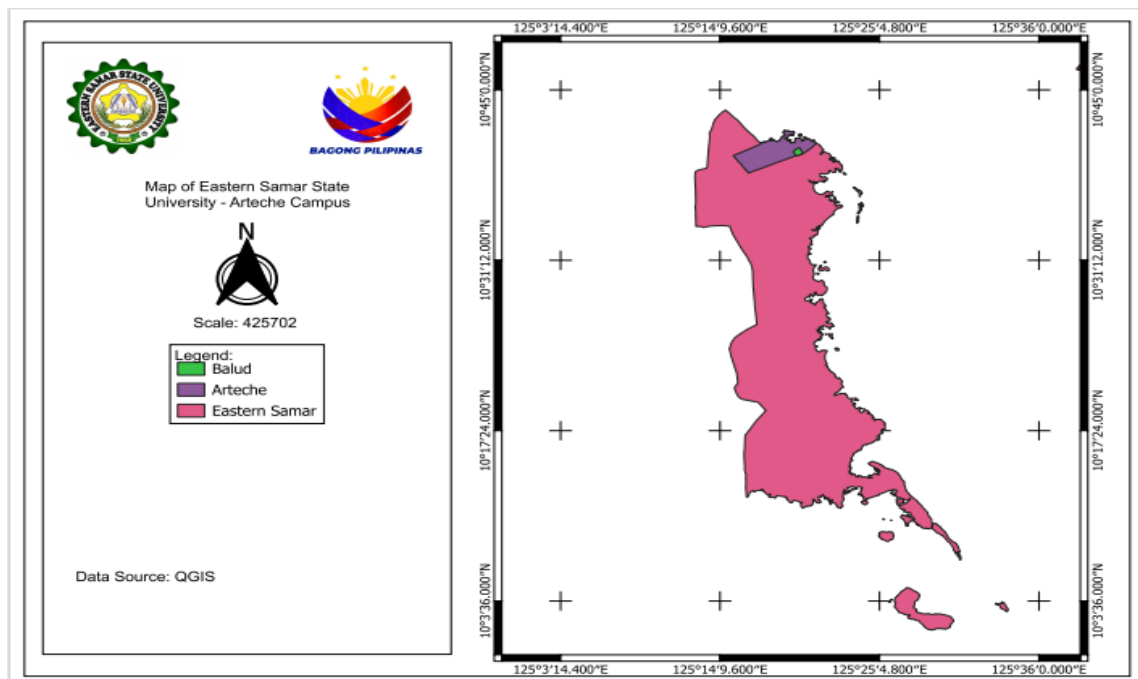


Figure 34. Map of ESSU ARTECHE



Respondents

This study has two types of respondents: 7 IT Professors and 283 BSIT students from the College of Computer Studies of Eastern Samar State University - Arteche Campus. The researchers' weighted mean will be used to measure each group's overall agreement with the survey statements. Slovin's formula will be applied by the researchers to determine the sample population of BSIT student respondents.

Slovin's Formula

$$n = \frac{N}{1 + Ne^2}$$

Where:

n = sample size

N = total BSIT student population (283)

e = margin of error (0.05)

$$n = \frac{283}{1 + 283(0.05)^2}$$

$$n = \frac{283}{1 + 0.7075}$$

$$n = \frac{283}{1.7075}$$

$$n = 165.74 \text{ or } 166$$

The researchers employed Slovin's Formula to calculate the sample population of BSIT students, resulting in a total of 166 samples. They utilized stratified sampling to divide these 166 samples among the four-year levels of the BSIT program:

$$n_i = (N_i/N) \times n$$

Where:

n_i = sample size per year level

N_i = population per year level

N = total BSIT population (283)

n = total BSIT sample (166)

Calculated allocations:

First Year:



$$n_i = (93/283) \times 166 \quad n_i = 0.3286 \times 166$$

$$n_i = 54.55 \text{ rounded to } 55$$

Second Year:

$$n_i = (71/283) \times 166 \quad n_i = 0.2509 \times 166$$

$$n_i = 41.65 \text{ rounded to } 42$$

Third Year:

$$n_i = (76/283) \times 166 \quad n_i = 0.2686 \times 166$$

$$n_i = 44.59 \text{ rounded to } 45$$

Fourth Year:

$$n_i = (43/283) \times 166 \quad n_i = 0.1519 \times 166$$

$$n_i = 25.22 \text{ adjusted rounded to } 24$$

The total sample population for BSIT student respondents consists of 55 from First Year, 42 from Second Year, 45 from Third Year, and 24 from Fourth Year, summing to 166 respondents.

Table 9. BSIT Student Respondents.

Year Level	Number of Students	Sample Size
1st Year	93	55
2nd Year	71	42
3rd Year	76	45
4th Year	43	24
Total	283	166

Table 10. Respondent (IT Professor Respondents)

Department	Number of IT Professors	Sample Size
College of Computer Studies	7	7
Total	7	7



Table 11. Respondents (Overall Respondents)

Category	Number	Sample Size
BSIT Students	283	166
IT Professors	7	7
Total	290	173

This table presents the distribution of respondents used in the evaluation of the proposed system. Complete enumeration was conducted for all IT Professors to ensure expert-based evaluation of system functionality and technical validity. For BSIT students, a probability sample was determined using Slovin's formula to ensure proportional representation across the different year levels of the BSIT program.

Instrumentation

A structured questionnaire will be used as the primary data-collection instrument. The instrument will be aligned with the ISO/IEC 25010-based quality characteristics identified in the study objectives. Each characteristic will be measured using a five-point Likert scale ranging from 5 (Strongly Agree) to 1 (Strongly Disagree). The questionnaire will be reviewed by IT Professors to ensure validity and alignment with the objectives of the study.

Data Gathering Procedure

The evaluation will be conducted within the College of Computer Studies laboratory environment. The system will first undergo unit and integration testing to verify technical correctness. After validation, it will be deployed in a controlled laboratory session where IT Professors and selected BSIT students will use the monitoring and control features of the system. Following the session, respondents will complete the structured questionnaire to assess the system according to the defined quality characteristics. All activities will be conducted within the system's operational scope and LAN environment.

Statistical Tool Used

Quantitative data collected from the evaluation questionnaire will be analyzed using the Weighted Mean to determine the level of agreement for each system quality characteristic. The weighted mean will provide a numerical representation of overall



respondent perception per evaluation category. Percentage distribution may also be computed to determine the proportion of responses across each scale level.

Table 12. The Five-Point Likert Scale

Scale	Range	Interpretation
5	4.21 - 5.00	Strongly Agree
4	3.42 - 4.20	Agree
3	2.61 - 3.40	Neutral
2	1.81 - 2.60	Disagree
1	1.0 - 1.80	Strongly Disagree

Table 13. Questionnaire Parameter

Quality Characteristic	Range	Interpretation
Functional Suitability	4.21 - 5.00	Highly Functional
	3.41 - 4.20	Functional
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Functional
	1.00 - 1.80	Not Functional
Performance Efficiency	4.21 - 5.00	Highly Efficient
	3.41 - 4.20	Efficient
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Efficient
	1.00 - 1.80	Not Efficient
Compatibility	4.21 - 5.00	Highly Compatible



	3.41 - 4.20	Compatible
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Compatible
	1.00 - 1.80	Not Compatible
Interaction Capability	4.21 - 5.00	Highly Capable
	3.41 - 4.20	Capable
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Capable
	1.00 - 1.80	Not Capable
Reliability	4.21 - 5.00	Highly Reliable
	3.41 - 4.20	Reliable
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Reliable
	1.00 - 1.80	Not Reliable
Security	4.21 - 5.00	Highly Secured
	3.41 - 4.20	Secured
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Secured
	1.00 - 1.80	Not Secured
Maintainability	4.21 - 5.00	Highly Maintainable
	3.41 - 4.20	Maintainable



	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Maintainable
	1.00 - 1.80	Not Maintainable
Flexibility	4.21 - 5.00	Highly Flexible
	3.41 - 4.20	Flexible
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Flexible
	1.00 - 1.80	Not Flexible
Safety	4.21 - 5.00	Highly Safe
	3.41 - 4.20	Safe
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Safe
	1.00 - 1.80	Not Safe

Data Analysis

The evaluation results will be analyzed using the Weighted Mean to determine the level of agreement for each quality characteristic. The Five-Point Likert Scale will serve as the basis for interpreting respondent feedback. The computed results will determine whether the system meets its intended objectives within the laboratory setting.

The formula for computing the weighted mean is as follows:

$$X = \frac{\sum Ns}{n}$$

Where:



X = Weighted Mean

N = Sum of weighted scores

s = Scale value

n = Number of respondents

Survey percentages may also be computed to determine the proportion of responses per category using the formula:

$$P = \frac{n}{t} \times 100$$

Where:

P = Percentage

n = Number of responses

t = Total number of respondents

Table 14. Total Grand Mean

Total Grand Mean	Range	Interpretation
	4.21 - 5.00	Highly Accepted
	3.41 - 4.20	Accepted
	2.61 - 3.40	Neutral
	1.81 - 2.60	Less Accepted
	1.00 - 1.80	Not Accepted

Unit Testing

Unit testing will be conducted to verify the functional correctness of each system component, including the Admin Monitoring Application, student PC Monitoring Program, and ESP32 Sensor Module. Each module will be tested independently using predefined test cases to ensure proper execution of monitoring, control, session management, and telemetry functions.



Integration Testing

Integration testing will be performed after unit validation to ensure that all system components operate cohesively within the local area network environment. The testing will verify proper communication between the admin monitoring dashboard, student workstations, and sensor modules, ensuring accurate data transmission and synchronized system behavior.

System Testing (Alpha Testing)

System testing will be conducted by IT Professors to assess the overall functionality, stability, and operational readiness of the system within a controlled laboratory session.

Acceptance Testing (Beta Testing)

Acceptance testing will be conducted with selected BSIT student respondents to evaluate the system based on the identified ISO/IEC 25010 quality characteristics. The objective of this phase is to determine whether the system satisfies user expectations and operational requirements within the defined laboratory scope.

Implementation

The implementation phase involves the structured installation and configuration of the IoT-Based Smart Laboratory Management System in the College of Computer Studies laboratory. The instructor's monitoring application will be installed on the instructor's computer, and the student application will be deployed on each lab workstation. Each workstation will also be equipped with an ESP32-based sensor module mounted adjacent to the PC. All devices will be connected to the campus LAN, and network settings (such as IP addressing and firewall rules) will be configured to allow communication between the sensor nodes, student workstations, and the instructor's dashboard. The instructor's PC should run a compatible operating system (Windows 10/11 with Python 3.13 or higher) to host the dashboard application, and each student PC should run a similar environment to host the client application. Necessary software dependencies and drivers for the ESP32 and sensors will be installed using the Arduino IDE or appropriate libraries, ensuring stable operation of the hardware components. Any calibration or initial configuration of sensor modules will be performed at setup to ensure accurate measurements and connectivity.

Once installed, the system will be used during laboratory sessions as follows: Instructors will initiate and manage sessions through the dashboard, and students will log



in at their workstations using the student application. The instructor can monitor each student's screen in real time, send commands (e.g. lock or unlock workstations, extend session time), and oversee lab progress. Session-based timers will enforce time limits as defined by the instructor. The system will display real-time workstation health metrics (such as temperature or fan status) on the dashboard for each station. All operation occurs within the on-premises lab environment; no external Internet or cloud services are required. The deployment environment includes a stable power supply and the university's LAN infrastructure. This controlled environment ensures that the system functions effectively using the specified hardware and software configurations without reliance on remote services.



RESEARCH QUESTIONNAIRE

Evaluation of the IoT-Based Smart Laboratory Management System

TITLE: IOT-BASED SMART LABORATORY MANAGEMENT SYSTEM FOR ESSU ARTECHE CAMPUS

Name: _____

Year & Section: _____ *(for students only)*

Respondent Type: ☐ IT Professor ☐ BSIT Student

Instructions:

Please rate each statement based on your experience with the developed system using the scale below. This evaluation aims to determine whether the system meets its intended objectives based on ISO/IEC 25010 standards. All responses will be kept confidential.

Rating Scale:

- 5 - Strongly Agree
- 4 - Agree
- 3 - Neutral
- 2 - Disagree
- 1 - Strongly Disagree

FUNCTIONALITY						
Category	Question	5	4	3	2	1
Completeness	The system allows instructors to view and monitor multiple student workstations in real time through a centralized dashboard.					
	The system displays complete monitoring					



	data including workstation activity and session information.					
Correctness	The system accurately controls workstation access such as locking and unlocking.					
	The session-based timer properly regulates workstation usage during laboratory sessions.					
Appropriateness	The system provides accurate temperature and fan status information for each workstation.					

EFFICIENCY						
Category	Question	5	4	3	2	1
Time Behavior	The system provides real-time monitoring with minimal delay.					
	The system executes control commands quickly.					
Resource Utilization	The system operates efficiently without excessive resource consumption.					



	The system performs smoothly during simultaneous monitoring of multiple workstations.					
Capacity	The system can handle multiple student workstations at the same time without performance issues.					

COMPATIBILITY						
Category	Question	5	4	3	2	1
Co-existence	The system operates properly within the local area network (LAN) environment.					
	The system does not interfere with other applications running in the laboratory.					
Interoperability	The system effectively integrates with ESP32-based sensor modules.					
	The system allows proper communication between instructor and student workstations.					
Appropriateness	The system is compatible with existing laboratory computer setups.					



INTERACTION CAPABILITY						
Category	Question	5	4	3	2	1
Understandability	The centralized dashboard is clear and easy to understand.					
	The system interface clearly presents workstation activity and status.					
Learnability	The system is easy to learn for both instructors and students.					
	Users can operate the system with minimal training.					
Operability	The system provides easy-to-use controls for monitoring and management.					

RELIABILITY						
Category	Question	5	4	3	2	1
Maturity	The system operates consistently during laboratory sessions.					
	The system provides stable real-time monitoring without interruptions.					



Availability	The system is available when needed during class sessions.					
	The system maintains continuous operation within the LAN environment.					
Fault Tolerance	The system recovers properly from minor errors or interruptions.					

SECURITY						
Category	Question	5	4	3	2	1
Confidentiality	The system ensures secure login for students and administrators.					
	Unauthorized users cannot access the system.					
Integrity	The system maintains accuracy of session and monitoring data.					
	Data is protected from unauthorized modification.					



Accountability	The system records user activity and session logs properly.					
----------------	---	--	--	--	--	--

MAINTAINABILITY						
Category	Question	5	4	3	2	1
Modularity	The system is structured in a way that supports easy updates.					
	System components can be modified independently.					
Modifiability	The system can be improved or enhanced when needed.					
	The system supports future feature additions.					
Testability	The system can be tested easily to ensure proper functionality.					



FLEXIBILITY						
Category	Question	5	4	3	2	1
Adaptability	The system can adapt to different laboratory conditions.					
	The system allows configuration of settings such as session time and controls.					
Installability	The system can be configured based on laboratory requirements.					
	The system can be easily configured within the LAN environment.					
Replaceability	The system can be upgraded or replaced when needed.					

SAFETY						
Category	Question	5	4	3	2	1
Operational Safety	The system helps prevent misuse of laboratory computers.					
	The system supports safe monitoring of workstation usage.					



Hardware Safety	The system helps monitor temperature to prevent overheating.					
	The system provides awareness of workstation hardware status.					
Environment Safety	The system contributes to a safer laboratory environment.					

Comments and Suggestions (Optional):



Chapter IV

RESULT AND DISCUSSION

The evaluators were IT experts from Eastern Samar State University – Arteche Campus. The evaluation criteria were based on the ISO 9001 software product quality model.

Design

The system was successfully developed according to precise specifications to meet the required data needs. Its operational framework, combining both the output and user interface components, has been fully implemented.

Output and User Interface

The application effectively meets user requirements for both on-screen and printed outputs. The outputs are designed to provide clear, accurate, and meaningful information, while the user interface offers an intuitive and user-friendly experience, ensuring smooth interaction and functionality.



System Testing (Alpha Testing)

Alpha testing together is the first stage of thorough testing done to make sure a product meets its functional standards and business requirements. It involves checking the entire system from start to finish to confirm that everything works correctly and that all intended features perform as expected.

Table 18. Frequency and Percentage Distribution – Alpha Testing

PARAMETERS	IT FACULTY	5	4	3	2	1	TOTAL
Functional Suitability							
Performance Efficiency							
Compatibility							
Usability							
Reliability							
Security							
Maintainability							
Portability							
Total							
Percentage							



Bibliography

A. Published Books

- Alagasan, K., Alkawaz, M. H., Hajamydeen, A. I., & Alshekhly, M. N. A. (2021, August). A review paper on advanced attendance and monitoring systems. In *Proceedings of the 2021 IEEE 12th Control and System Graduate Research Colloquium (ICSGRC)* (pp. 195-200). IEEE. <https://doi.org/10.1109/ICSGRC53186.2021.9515249>
- Angamuthu, V., & Thiyagarajan, V. S. (2025). LAN Monitor Tool: An efficient real-time system tracking and USB control. *International Journal of Scientific Research in Science and Technology*, 12(4), 858-864.
- Bolasco, M. A. N., Costales, G. L., Saldivar, P. B. E., Valdez, C. D. C., & Cruz, R. D. L. (2025). LabTrack: A comprehensive approach to attendance tracking and computer usage monitoring. *International Journal of Research and Innovation in Applied Science*, 3(3), 257-260.
- Czekalski, P., Tokarz, K., Kuaban, G. S., Sell, R., Nikitenko, A., Berkolds, K., & Lipka, Ł. (2025). Design and implementation of a next-generation remote lab for IoT and Industry 4.0 education. *Procedia Computer Science*, 270, 387-396.
- Mohanty, A. K., Kumar, S. V. A., Thongam, R., Kawale, S. R., Jakkani, A. K., & Vani, V. G. (2024). Enhancing classroom engagement through IoT-enabled smart learning environments. *ShodhKosh: Journal of Visual and Performing Arts*, 5(1), 1003-1010.

B. Websites

- Faronics. (2026). Faronics Insight - Classroom management software. Retrieved from <https://www.faronics.com/products/insight> (Provides features of the Insight classroom management tool).
- LanSchool (Lenovo). (2021, June 22). How screen monitoring supports stronger student engagement. Retrieved from <https://lanschool.com/blog/features-and-updates/how-screen-monitoring-supports-stronger-student-engagement> (Describes LanSchool's screen monitoring and blanking features).



LabStats. (2020, August 7). Getting started with computer lab usage tracking software. Retrieved from <https://labstats.com/blogs/getting-started-with-computer-lab-usage-tracking-software/> (Explains lab usage tracking via client software on each PC).

NetSupport Ltd. (n.d.). Classroom management and monitoring with NetSupport School. Retrieved from <https://www.netsupportschool.com/features> (Details NetSupport School features such as multi-monitor viewing, application control, and screen blanking).

Veyon Solutions. (2026). Monitor and control all computers - Veyon. Retrieved from <https://veyon.io/en/> (Describes Veyon's open-source features: multi-screen view, remote control, and locking inputs).